THE ASTROPHYSICAL JOURNAL SUPPLEMENT SERIES, 124:241–263, 1999 September
© 1999. The American Astronomical Society. All rights reserved. Printed in U.S.A.

INTEGRATION OF NUCLEAR REACTION NETWORKS FOR STELLAR HYDRODYNAMICS

F. X. TIMMES
Center on Astrophysical Thermonuclear Flashes, University of Chicago, Chicago, IL 60637; fxt@burn.uchicago.edu
Received 1999 March 12; accepted 1999 April 8

ABSTRACT

Methods for solving the stiff system of ordinary differential equations that constitute nuclear reaction networks are surveyed. Three semi-implicit time integration algorithms are examined; a traditional first-order–accurate Euler method, a fourth-order–accurate Kaps-Rentrop method, and a variable-order Bader-Deuflhard method. These three integration methods are coupled to eight different linear algebra packages. Four of the linear algebra packages operate on dense matrices (LAPACK, LUDCMP, LEQS, GIFT), three of them are designed for the direct solution of sparse matrices (MA28, UMFPACK, Y12M), and one uses an iterative method for sparse matrices (BiCG). The scaling properties and behavior of the 24 combinations (3 time integration methods times 8 linear algebra packages) are analyzed by running each combination on seven different nuclear reaction networks. These reaction networks range from a hardwired 13 isotope α-chain and heavy-ion reaction network, which is suitable for most multidimensional simulations of stellar phenomena, to a 489 isotope reaction network, which is suitable for determining the yields of isotopes lighter than technetium in spherically symmetric models of Type II supernovae. Each of the time integration methods and linear algebra packages are capable of generating accurate results, but the efficiency of the various methods—evaluated across several different machine architectures and compiler options—differ dramatically. If the execution speed of reaction networks that contain less than about 50 isotopes is an overriding concern, then the variable-order Bader-Deuflhard time integration method coupled with routines generated from the GIFT matrix package or LAPACK with vendor-optimized BLAS routines is a good choice. If the amount of storage needed for any reaction network is a concern, then any of the sparse matrix packages will reduce the storage costs by 70%–90%. When a balance between accuracy, overall efficiency, and ease of use is desirable, then the variable-order Bader-Deuflhard time integration method coupled with the MA28 sparse matrix package is a strong choice.

Subject headings: hydrodynamics — methods: numerical — nuclear reactions, nucleosynthesis, abundances — stars: interiors

1. INTRODUCTION

Models of stellar events typically require the energy generated due to nuclear burning over a large span of temperatures, densities, and compositions. The average energy generated or lost over a period of time is found by integrating the set of differential equations that represent the abundances of the isotopes. In some stellar events, such as Type II supernova models, the abundances themselves are of primary interest. In either case, the coefficients that appear in the differential equations span many orders of magnitude because of exponential dependences on the temperature (see, for example, Arnett 1966). This behavior of the coefficients makes the differential equations that comprise a nuclear reaction network stiff. Solving a stiff set of differential equations is generally more involved than solving a nonstiff one. With over 10^9 calls to the nuclear reaction networks being common in two- and three-dimensional hydrodynamic models of stellar phenomena, it is very desirable to have the solution method be as efficient as possible and yet accurately represent the relevant physics.

The purpose of this paper is to compare the accuracy and execution speed of various methods for solving nuclear reaction networks. Three time integration algorithms, eight linear algebra packages, and seven reaction networks are encompassed by this survey. The three time integration methods are semi-implicit (semi-implicit, as opposed to implicit, because some linearization is done); a traditional first-order–accurate Euler method, a fourth-order–accurate Kaps-Rentrop method, and a variable-order Bader-Deuflhard method. Four of the linear algebra packages operate on dense matrices (LAPACK, LUDCMP, LEQS, GIFT), three of them perform direct solutions of sparse matrices (MA28, UMFPACK, Y12M), and one uses an iterative method for sparse matrices (BiCG). The most common combination in present use is probably the first-order Euler method and the LEQS matrix package. The seven reaction networks that will be used to examine the scaling properties of the various time integrators and linear algebra packages are (1) a hardwired 13 isotope α-chain and heavy-ion reaction network that is suitable for most multidimensional simulations of stellar phenomena, (2) a hardwired 19 isotope reaction network adds isotopes to accommodate hydrogen burning and photodisintegration, (3) a 47 isotope reaction network that relaxes various assumptions made for the 13 and 19 isotope reaction networks, (4) a 76 isotope reaction network, (5) a 127 isotope reaction network, (6) a 200 isotope reaction network, and (7) a 489 isotope reaction network that is suitable for determining the yields of isotopes lighter than technetium is spherically symmetric models of Type II supernovae. The motivation for undertaking this survey is to provide benchmark values for the accuracy and speed of reaction network solvers that are commonly used in models of various stellar phenomena, and to stimulate further improvement in the algorithms. These benchmarks allow an assessment of the reaction network solvers, and permit practical guidelines to be derived. Nuclear reaction network solvers that were not

encompassed by this survey may also benefit from comparisons to the benchmark values.

In § 2 the suite of nuclear reaction networks used to test the stiff ordinary differential equation integrators and linear algebra packages are discussed. The salient features of the time integration techniques are presented in § 3, the linear algebra packages are briefly discussed in § 4, and in § 5 the results of the efficiency tests are presented and analyzed. A summary of the findings is given in § 6, as are some pragmatic suggestions on which time integration method and linear algebra package to use in a given situation.

2. A SUITE OF SEVEN TEST REACTION NETWORKS

Let isotope i have Z_i protons and A_i nucleons (protons + neutrons). Let the aggregate total of isotope i have a number density n_i (in cm^{-3}) in material with temperature T (in K) and mass density ρ (in g cm^{-3}). Define the mass fraction (dimensionless) of isotope i as $X_i = \rho_i/\rho = n_i A_i/(\rho N_\mathrm{A})$, where N_A is Avogadro's number, and define the molar abundance of isotope i as $Y_i = X_i/A_i = n_i/(\rho N_\mathrm{A})$. Mass conservation is then expressed by

$$\sum_{i=1}^{N} X_i = 1 \; . \tag{1}$$

This expression should be checked at every time step in a numerical simulation of a stellar event. Failure to satisfy equation (1) to the limiting precision of the arithmetic may manifest itself in the unphysical buildup (or decay) of the abundances or energy generation rate. Models of events that are sensitive to the abundance levels of trace species (e.g., classical nova envelopes) may suffer inaccuracies if mass conservation is significantly violated over sufficiently long timescales.

The most general continuity equation for isotope i in a Lagrangian formulation is

$$\frac{dY_i}{dt} + \nabla \cdot (Y_i \boldsymbol{V}_i) = \dot{R}_i \; . \tag{2}$$

In this set of partial differential equations $\dot{R}_i$ is the total reaction rate due to all binary reactions of the form $i(j, k)l$:

$$\dot{R}_i = \sum_{j,k} Y_l Y_k \lambda_{kj}(l) - Y_i Y_j \lambda_{jk}(i) \; , \tag{3}$$

where λ_{kj} and λ_{jk} are the reverse (creation) and forward (destruction) nuclear reaction rates, respectively. Contributions from three-body reactions, such as the triple-α reaction, are easy to append to equation (3). The mass diffusion velocities V_i in equation (2) are obtained from the solution of a multicomponent diffusion equation (Chapman & Cowling 1970; Burgers 1969; Williams 1988), and reflect the fact that mass diffusion processes arise from pressure, temperature, and abundance gradients as well as external gravitational or electrical forces.

Consider the case where the mass diffusion velocities in equation (2) are set to zero. There are two reasons for considering this case. The first reason is that the physics of the situation may dictate that mass diffusion processes are small over the timescales of interest when compared to other transport process such as thermal diffusion or viscous diffusion (i.e., large Lewis numbers and/or small Prandtl numbers). Such a situation applies, for example, to the study of laminar flame fronts propagating through the interior of a white dwarf. The second reason for considering this case is that one may wish to apply the numerical technique of operator splitting. In operator splitting, the various pieces of the physics are decoupled from one another over a given time step. Each piece is then solved for independently of the others, with the various pieces being appropriately summed at the end of the time step. This approach is quite common in spherically symmetric stellar evolution programs and in multidimensional hydrodynamic programs. Typically, for example, the properties of the adiabatic fluid flow are determined for a given time step. The energy generated or changes to the composition due to thermonuclear reactions over the time step are then determined. These two pieces are then summed. Potential disadvantages of the operator splitting technique are that the point at which decoupling becomes a poor approximation is difficult to estimate a priori, and the ordering of the operators may not be commutative. For either reason, because the physics allows it or because operator splitting is being used, setting the mass diffusion velocity to zero transforms equation (2) into

$$\frac{dY_i}{dt} = \dot{R}_i \; . \tag{4}$$

This set of ordinary differential equations may be written in the more compact and standard form

$$\dot{\boldsymbol{y}} = \boldsymbol{f}(\boldsymbol{y}) \; . \tag{5}$$

The set of ordinary differential equations in equation (4) or equation (5) are what constitute a "reaction network." In the absence of coupling to neighboring regions through mass diffusion processes, any changes to a composition are due to local processes.

The coefficients that appear in equation (4) or equation (5) span many orders of magnitude because the nuclear reaction rates have highly nonlinear dependences on the temperature, and because the abundance levels themselves typically range over several orders of magnitude (e.g., the solar isotopic abundances). As a result, the differential equations that comprise a nuclear reaction network are "stiff." From a mathematical point of view a set of equations is stiff when the ratio of the maximum to the minimum eigenvalue of the Jacobian matrix $\tilde{\boldsymbol{J}} = \partial \boldsymbol{f}/\partial \boldsymbol{y}$ is large and imaginary. From a physics point of view, "stiff" means that at least one of the isotopic abundances is changing on a much faster timescale than the abundance level of another isotope. From a practical point of view, "stiff" generally means that an implicit time integration, hence having the Jacobian matrix $\tilde{\boldsymbol{J}}$, is necessary. Solving a stiff set of differential equations with implicit methods is always more involved than solving a nonstiff set of differential equations with explicit methods.

Seven nuclear reaction networks were chosen as test vehicles to assess the scaling properties of the implicit time integrators and linear algebra packages on nuclear reaction networks. Before discussing this test suite, it is instructive to look at an example of how equation (4) and the associated Jacobian matrix are formed.

Consider the $^{12}\mathrm{C}(\alpha, \gamma)^{16}\mathrm{O}$ reaction, which competes with the triple-α reaction during helium burning in stars. The rate at which this reaction proceeds, call it R, is critical for evolutionary models of massive stars, since it determines how much of the core is carbon and how much of the core is oxygen after helium is exhausted. This reaction sequence contributes to the right-hand side of equation (5) through

the terms

$$\dot{Y}(^4\mathrm{He}) = -Y(^4\mathrm{He})Y(^{12}\mathrm{C})R + \cdots ,$$
$$\dot{Y}(^{12}\mathrm{C}) = -Y(^4\mathrm{He})Y(^{12}\mathrm{C})R + \cdots , \quad (6)$$
$$\dot{Y}(^{16}\mathrm{O}) = +Y(^4\mathrm{He})Y(^{12}\mathrm{C})R + \cdots ,$$

where the ellipses indicate additional terms coming from other reaction sequences. The minus signs indicate that helium and carbon are being destroyed, and the positive sign indicates that oxygen is being created. Each of the three expressions in equation (6) contributes two terms to the Jacobian matrix $\tilde{J} = \partial f/\partial y$:

$$J(^4\mathrm{He}, ^4\mathrm{He}) = -Y(^{12}\mathrm{C})R + \cdots ,$$
$$J(^4\mathrm{He}, ^{12}\mathrm{C}) = -Y(^4\mathrm{He})R + \cdots , \quad (7a)$$
$$J(^{12}\mathrm{C}, ^4\mathrm{He}) = -Y(^{12}\mathrm{C})R + \cdots ,$$
$$J(^{12}\mathrm{C}, ^{12}\mathrm{C}) = -Y(^4\mathrm{He})R + \cdots , \quad (7b)$$
$$J(^{16}\mathrm{O}, ^4\mathrm{He}) = +Y(^{12}\mathrm{C})R + \cdots ,$$
$$J(^{16}\mathrm{O}, ^{12}\mathrm{C}) = +Y(^4\mathrm{He})R + \cdots . \quad (7c)$$

Entries in the Jacobian matrix represent the flow, in nuclei per second into (positive) or out of (negative) an isotope. All of the temperature and the density dependences are included in the reaction rate R. The Jacobian matrices that arise from nuclear reaction rate R. The Jacobian matrices that arise from nuclear reaction networks are not positive definite or symmetric, since the forward and reverse reaction rates are generally not equal. While the nonzero pattern of the Jacobian matrix does not change with time, the magnitude of the matrix entries change as either the abundances, temperature, or density changes with time. Physically the Jacobian matrix should be completely dense, since every isotope can, in principle, react with every other isotope. The sparse structure of the Jacobian matrix comes from ignoring the neglectable. For example, there are no reaction rates or matrix elements for $^{28}\mathrm{Si} + ^{28}\mathrm{Si}$ because at $\sim 5 \times 10^9$ K, where the ^{28}Si nuclei would have enough kinetic energy to overcome the large Coulomb barrier, it is far more likely that the silicon nuclei melt by a sequence of photodisintegration reactions into α-particles, neutrons, and protons (but primarily α-particles).

The first reaction network in the test suite is a 13 isotope α-chain plus heavy-ion reaction network, which is suitable for most multidimensional simulations of stellar phenomena where having a reasonably accurate energy generation rate is of primary concern. The Jacobian matrix for this α-chain network during the start of a vigorous helium-burning episode is shown in Figure 1. Nonzero entries in the matrix are color-coded according to the magnitude of the flow rate into (red tones) or out of (green-blue tones) an isotope. Matrix elements which are zero are colored white. The matrix is organized and oriented so that the $J(1, 1)$ entry corresponds to $J(^4\mathrm{He}, ^4\mathrm{He})$ in the upper left corner, as indicated by the row and column labels. The number of reaction rates in the network, along with the percentage of the matrix that is zero, is shown to the left of the Jacobian matrix.

The second network in the test suite contains 19 isotopes. It has the same α-chain and heavy-ion reactions as the 13 isotope reaction network, but includes additional isotopes to accommodate hydrogen burning and some aspects of photodisintegration. This reaction network is described in some detail by Weaver, Zimmerman, & Woosley (1978).

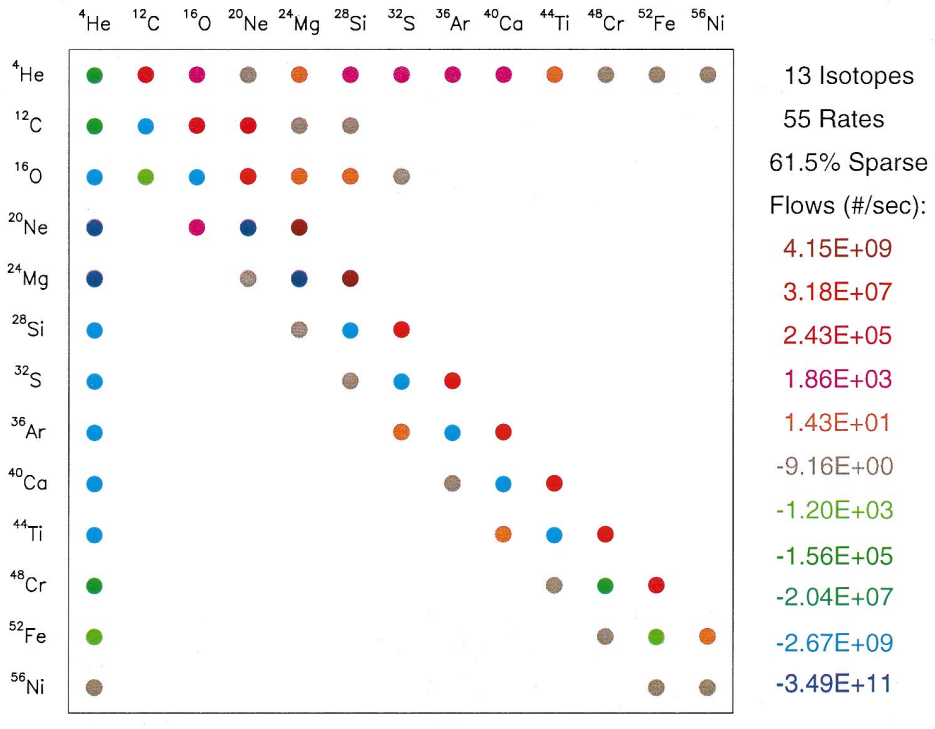

FIG. 1.—Jacobian matrix for the 13 isotope reaction network at a time shortly after the onset of hydrostatic helium burning. Labels on the top and left axes give the names of the isotopes in the reaction network, and the matrix is oriented so that the (1, 1) entry corresponds to (^{4}He, ^{4}He) in the upper left corner. Nonzero entries in the matrix are color-coded according to the magnitude of the flow rate into (*red tones*) or out of (*blue tones*) an istope. The nonzero pattern of the matrix does not change with time, but the magnitudes of the matrix entries change as either abundances, temperature, or density changes in time.

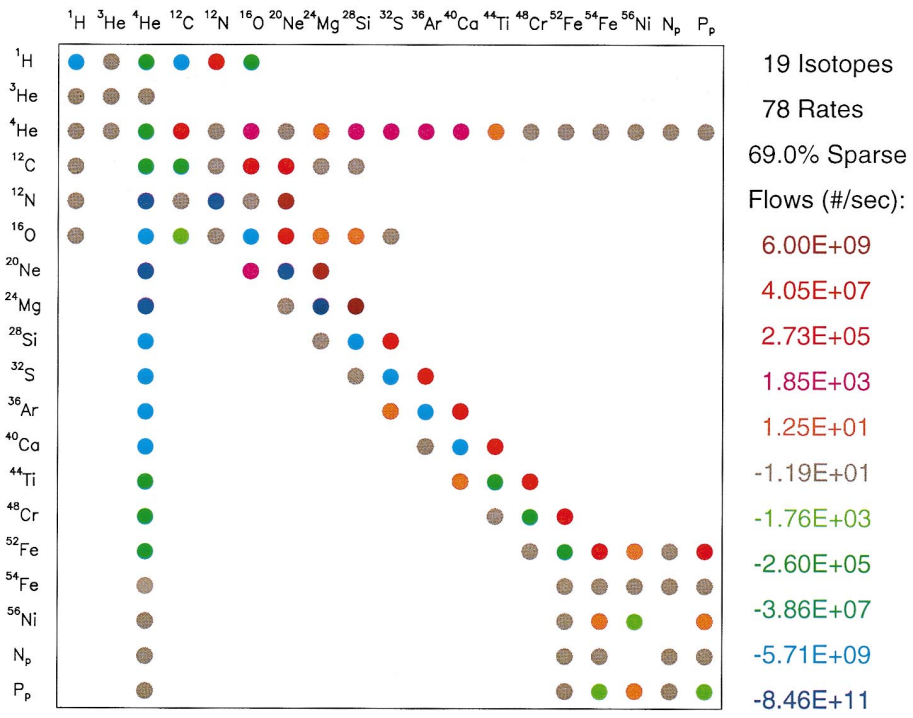

FIG. 2.—Jacobian matrix for the 19 isotope reaction network at a time shortly after the onset of helium burning. The format is the same as that used in Fig. 1. The number of reaction rates in the reaction network and the percentage of the matrix that is zero are given in the legend to the left of the figure. Both the 13 and 19 isotope reaction networks are hardwired, where the reaction sequences are carefully entered by hand—an approach that is suitable for small reaction networks.

Figure 2 shows the Jacobian matrix for this network at the onset of a vigorous helium-burning episode at a temperature of 3×10^9 K and a density of 2×10^8 g cm^{-3}. The color coding of the matrix entries, orientation of the matrix, and labeling are the same as in Figure 1. Both the 13 and 19 isotope reaction networks are examples of "hardwired" reaction networks, where each of the reaction sequences is carefully entered by hand (e.g., the sequence given by eqs. (6)

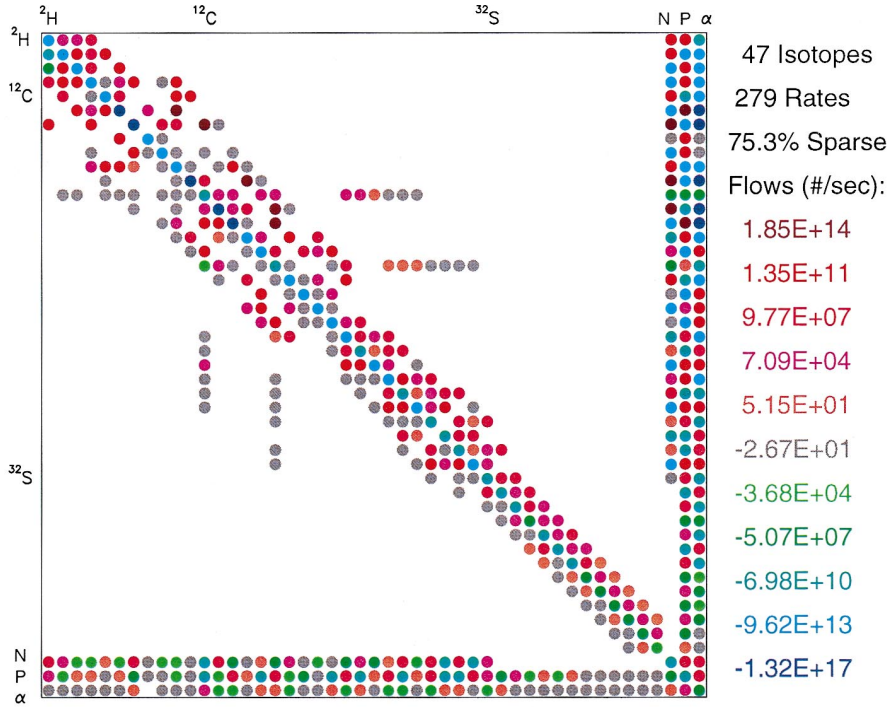

FIG. 3.—Jacobian matrix for the 47 isotope reaction network at a time shortly after the onset of helium burning. Labels for the locations of a few of the isotopes in the reaction network are given on the left and top axes. The format is the same as that used in Fig. 1. This reaction network is softwired, where all of the right-hand sides of the stiff ordinary differential equations and their Jacobian matrix entries have been constructed by a system of integer pointers for all reaction sequences. Neutrons, protons, and α-particles are placed last in the isotope list, which yields the characteristic arrowhead pattern.

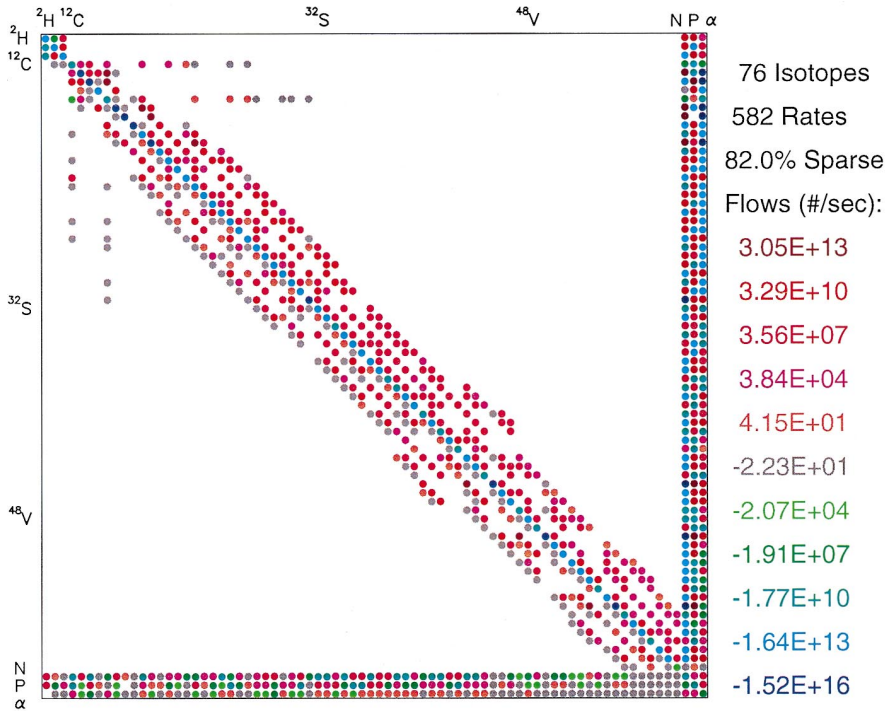

FIG. 4.—Jacobian matrix for the 76 isotope reaction network at a time shortly after the onset of a hydrostatic helium-burning episode at a temperature of 3×10^9 K and a density of 2×10^8 g cm^{-3}. The format of the figure is the same as in Fig. 1.

and (7). This hardwired approach is suitable for small networks when minimizing the CPU time required to run the reaction network is a primary concern. A disadvantage of hardwired reaction networks is their lack of flexibility.

The third network in the test suite contains 47 isotopes. It is functionally the same as the 19 isotope reaction network, but relaxes various assumptions about the relative rate of (p, γ) to (α, γ) reactions. The Jacobian matrix for the 47 isotope network at a time shortly after the beginning of hydrostatic helium burning is shown in Figure 3. The matrix is organized and oriented so that the $J(1, 1)$ entry is in the upper left corner, and the locations of a few isotopes present in the reaction network are labeled along the left and top axis.

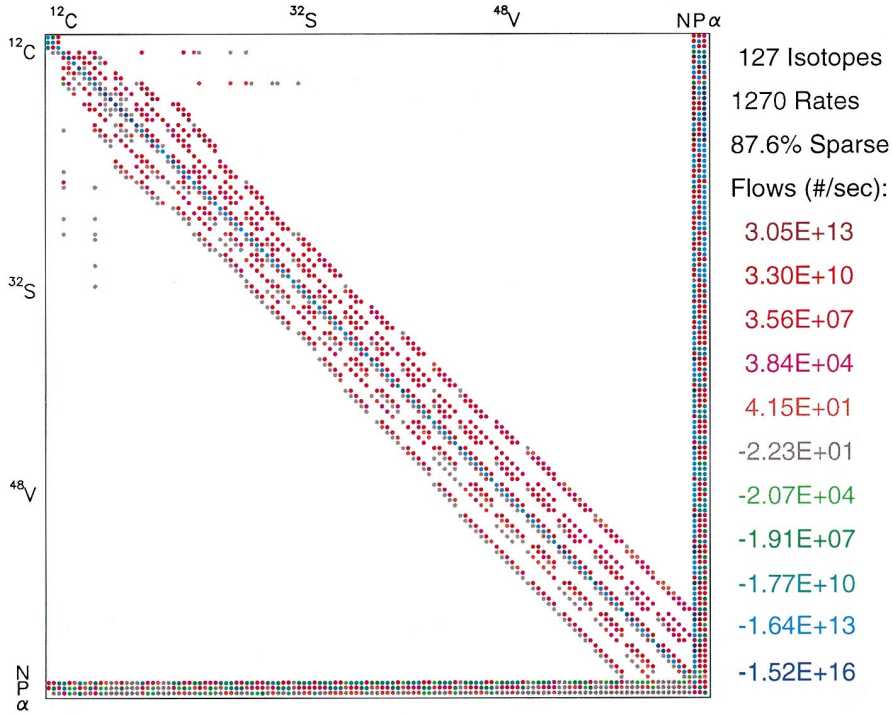

FIG. 5.—Jacobian matrix for the 127 isotope reaction network at a time shortly after the onset of hydrostatic helium burning. The format of the figure is the same as in Fig. 1. The number of reaction rates included in the reaction network and the percentage of the matrix elements that are zero are shown to the left of the figure.

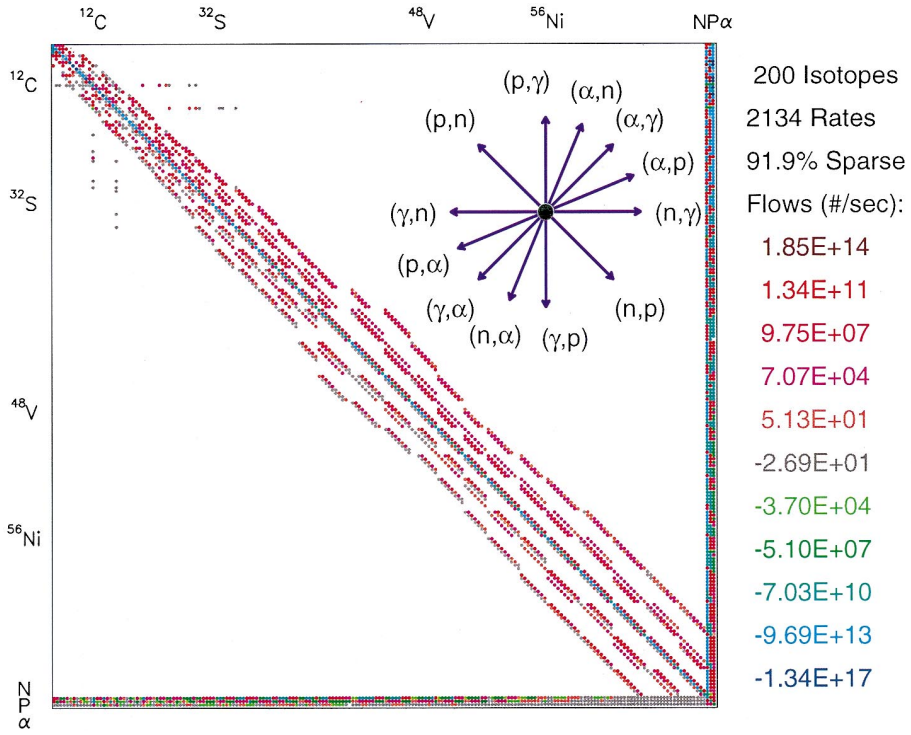

FIG. 6.—Jacobian matrix for the 200 isotope reaction network at a time shortly after the onset of helium burning. The format of the figure is the same as in Fig. 1. A schematic of the 14 fundamental binary reactions appears in the upper right corner.

The 47 isotope reaction network is a "softwired" reaction network. In this approach, all of the right-hand sides and Jacobian matrix entries (e.g., eqs. (5) and (6) for all reaction sequences are constructed automatically by a system of pointers. One needs only to supply a list of the isotopes to be included in the reaction network. The disadvantage of this complete generality is that the execution speed of a softwired reaction network is (typically) slightly

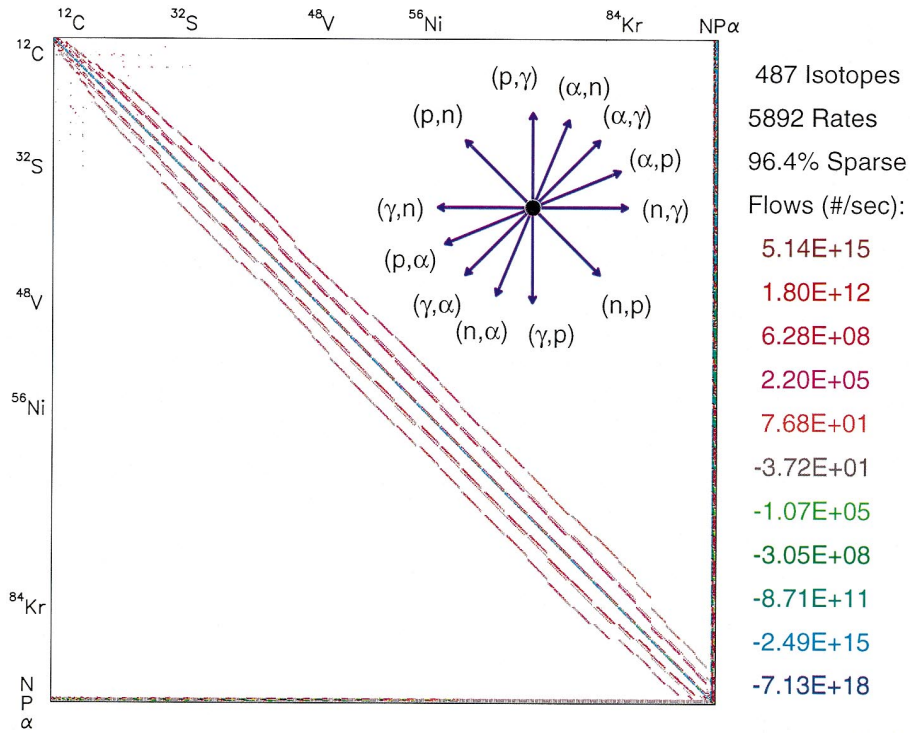

FIG. 7.—Jacobian matrix for the 489 isotope reaction network at a time shortly after the onset of helium burning. The format of the figure is the same as in Fig. 1, and a schematic of the 14 basic binary reactions is in the upper right corner. Note that this Jacobian matrix is quite sparse, i.e., a large fraction of matrix elements are zero. Storing all those zeros or performing arithmetic operations on them during a calculation would be a waste of resources.

slower than an equivalent hardwired one. The softwired reaction networks always include neutrons, protons, and α-particles, since these nucleons are present in just about all binary reaction sequences. This is why Figure 3, and all of the softwired reaction networks discussed below, have the characteristic arrowhead pattern. Putting the neutrons, protons, and α-particles first (up-arrow configuration) or last (down-arrow configuration) makes no difference physically, but it has consequences for some of the linear algebra packages, which are discussed in § 5.

The fourth network in the test suite has 76 isotopes, the fifth network contains 127 isotopes, and the sixth network contains 200 isotopes. The Jacobian matrices for these reaction networks during the early phases of hydrostatic helium burning at a temperature of 3×10^9 K and density of 2×10^8 g cm^{-3} are shown in Figures 4, 5, and 6, respectively. A schematic of the 14 fundamental binary reactions is given in the upper right corner of Figure 6. The seventh and final network in the test suite contains 489 isotopes, and is suitable for determining the yields of most isotopes lighter than technetium in spherically symmetric models of Type II supernovae. This reaction network is described in some detail by Woosley & Hoffman (1992) and Woosley & Weaver (1995). The Jacobian matrix for this reaction network during a hydrostatic helium-burning run is shown in Figure 7, where a schematic of the basic binary reactions is in the upper right corner. All of these reaction networks are softwired in the same manner as described above.

Figures 1–7 show that the Jacobian matrices of nuclear reaction networks become more sparse (a larger fraction of zero matrix elements) as the number of isotopes in a reaction network is increased. It makes little sense to either store all those zeros in memory or perform arithmetic on them during a calculation.

3. THREE SEMI-IMPLICIT INTEGRATION METHODS

The first method investigated in this survey for integrating the reaction networks in equation (5) is the simple first-order Euler method, which advances the system over a time step h according to

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \boldsymbol{\Delta} \, . \tag{8}$$

The change in the abundances $\boldsymbol{\Delta}$ is obtained from expanding the right-hand side of $\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \boldsymbol{f}(\boldsymbol{y}_{n+1})$ to first-order about the known $\boldsymbol{f}(\boldsymbol{y}_n)$. One finds

$$(\tilde{\mathbf{1}}/h - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta} = \boldsymbol{f}(\boldsymbol{y}_n) \, , \tag{9}$$

which is simply the matrix equation $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} = \boldsymbol{b}$. Thus, the first-order Euler method requires one evaluation of the Jacobian matrix, one evaluation of the right-hand side, one matrix decomposition, and one back-substitution. This method has a smaller cost per successful time step than any of the other methods investigated and is probably the most common time integration method for evolving reaction networks. However, the first-order Euler method provides no estimate of how accurate the integration step was! One does not know whether the time step was accurate to 1 part in 10^{-2} or even worse. Heuristics are typically invoked in order to gain some false sense of accuracy. Such heuristics are almost always based upon limiting the change in any abundance that is above some level to less than some specified value (say, 5%). While nuclear reaction networks can be integrated in this manner, and commonly are, one still has no estimate of how accurate the integration is. The first-order Euler method described above could be modified to do "step doubling" in order to gain some formal accuracy estimate: take two half time steps and one full time step. If the two abundance vectors do not agree within some specified accuracy tolerance, then reject the time step, reduce the time step by some factor, and try again. Step-doubling techniques have a significantly greater cost than either of the two integration methods that are described next.

The second integration method investigated in this survey is the fourth-order–accurate Kaps-Rentrop method. In essence, this method is an implicit Runge-Kutta algorithm. The stiff ordinary differential equations in equation (5) are advanced over a time step h according to

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \sum_{i=1}^{4} b_i \boldsymbol{\Delta}_i \, , \tag{10}$$

where the four vectors $\boldsymbol{\Delta}_i$ are found from successively solving the four matrix equations

$$\begin{aligned}
(\tilde{\mathbf{1}}/\gamma h - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta}_1 &= \boldsymbol{f}(\boldsymbol{y}_n) \, , \\
(\tilde{\mathbf{1}}/\gamma h - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta}_2 &= \boldsymbol{f}(\boldsymbol{y}_n + a_{21} \boldsymbol{\Delta}_1) + c_{21} \boldsymbol{\Delta}_1/h \, , \\
(\tilde{\mathbf{1}}/\gamma h - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta}_3 &= \boldsymbol{f}(\boldsymbol{y}_n + a_{31} \boldsymbol{\Delta}_1 + a_{32} \boldsymbol{\Delta}_2) \\
&\quad + (c_{31} \boldsymbol{\Delta}_1 + c_{32} \boldsymbol{\Delta}_2)/h \, , \\
(\tilde{\mathbf{1}}/\gamma h - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta}_4 &= \boldsymbol{f}(\boldsymbol{y}_n + a_{31} \boldsymbol{\Delta}_1 + a_{32} \boldsymbol{\Delta}_2) \\
&\quad + (c_{41} \boldsymbol{\Delta}_1 + c_{42} \boldsymbol{\Delta}_2 + c_{43} \boldsymbol{\Delta}_3)/h \, .
\end{aligned} \tag{11}$$

The b_i, γ, a_{ij}, and c_{ij} in equations (10) and (11) are fixed constants of the method. An estimate of the accuracy of the integration step is made by comparing a third-order solution with a fourth-order solution, which is a significant improvement over the basic Euler method. The minimum cost of this method—which applies for a single time step that meets or exceeds the specified integration accuracy—is one Jacobian evaluation, three evaluations of the right-hand side, one matrix decomposition, and four back-substitutions. The nominal cost of this method over the Euler method is quite small: two extra evaluations of the right-hand side and three extra back-substitutions. Possessing an estimate of the integration accuracy seems well worth the small additional cost. Note that equation (11) represents a staged set of linear equations ($\boldsymbol{\Delta}_4$ depends on $\boldsymbol{\Delta}_3$ ... depends on $\boldsymbol{\Delta}_1$). Not all of the right-hand sides are known in advance. This general feature of the higher order integration methods examined in this survey will impact the optimal choice of a linear algebra package. The fourth-order Kaps-Rentrop routine used in this survey is a combination of the routine GRK4T given by Kaps & Rentrop (1979) and the routine STIFF given by Press et al. (1996).

The third integration method investigated in this survey is the variable-order Bader-Deuflhard method. The reaction network expressed in equation (5) is advanced over a large time step H from $\boldsymbol{Y}_n$ to $\boldsymbol{y}_{n+1}$ by the following sequence of matrix equations. First,

$$\begin{aligned}
h &= H/m \, , \\
(\tilde{\mathbf{1}} - \tilde{\boldsymbol{J}}) \cdot \boldsymbol{\Delta}_0 &= h \boldsymbol{f}(\boldsymbol{y}_n) \, , \\
\boldsymbol{y}_1 &= \boldsymbol{y}_n + \boldsymbol{\Delta}_0 \, .
\end{aligned} \tag{12}$$

Then, from $k = 1, 2, \ldots, m - 1$,

$$(\tilde{1} - \tilde{J}) \cdot x = hf(y_k) - \Delta_{k-1} ,$$
$$\Delta_k = \Delta_{k-1} + 2x ,$$
$$y_{k+1} = y_k + \Delta_k , \quad (13)$$

and closure is obtained by the last stage

$$(\tilde{1} - \tilde{J}) \cdot \Delta_m = h[f(y_m) - \Delta_{m-1}] ,$$
$$y_{n+1} = y_m + \Delta_m . \quad (14)$$

This staged sequence of matrix equations is executed at least twice with $m = 2$ and $m = 6$, which yields a fifth-order method at minimum. The sequence may be executed a maximum of 7 times, which yields a 15th-order method. The exact number of times the staged sequence is executed depends on the accuracy requirements and the smoothness of the solution. Estimates of the accuracy of an integration step are made by comparing the solutions derived from different orders. The minimum cost of this method—which applies for a single time step that meets or exceeds the specified integration accuracy—is one Jacobian evaluation, eight evaluations of the right-hand side, two matrix decompositions, and ten back-substitutions. This minimum cost can be increased at a rate of one decomposition (the expensive part) and m back-substitutions (the inexpensive part) for every increase in the order $2k + 1$. The cost of increasing the order is compensated for, hopefully, by taking a correspondingly larger (but accurate) time step. The controls for order versus step size are a built-in part of the Bader-Deuflhard method. The cost per step of this integration method is at least twice as large as the cost per step of either the Euler or the Kaps-Rentrop method. However, if the Bader-Deuflhard method can take accurate time steps that are at least twice as large, then this method will be more efficient globally. Note that in equations (12)–(14), as in equation (11), not all of the right-hand sides are known in advance, since the sequence of linear equations is staged. This staging feature of the integration method will make some matrix packages a more efficient choice. The variable-order Bader-Deuflhard routine used in this survey is a combination of the routine METANI given by Bader & Deuflhard (1983) and the routine STIFBS given by Press et al. (1996).

All three of the semi-implicit time integration methods described above require the solution of a set of linear algebraic equations $\tilde{A} \cdot x = b$, which generally (but not always) dominates the total cost of a single time step. The matrix is generally of the form $\tilde{A} = \tilde{1} - h\tilde{J}$, which for nuclear reaction networks yields diagonally dominant matrices (see Figs. 1–7). This property of the reaction network matrices can be used to good advantage in some of the linear algebra packages that are now described.

4. EIGHT LINEAR ALGEBRA PACKAGES

Each of the three semi-implicit time integration methods described in the preceding section was executed in conjunction with eight linear algebra packages. Four of the packages operate on full matrices $A(i, j)$: LAPACK, LUDCMP, LEQS, and GIFT. Although the latter two packages take some advantage of zero matrix elements, they still require the storage of a full matrix and are thus categorized with the other dense-matrix packages. Three of the linear algebra packages perform direct solutions of sparse sets of linear equations: MA28, UMFPACK, and Y12M. One matrix package generates the solution to sparse systems by an iterative method: BiCG. All the sparse-matrix packages only store the nonzero elements of the matrix and do not (by definition) operate on matrix elements that are zero. Each of the linear algebra packages used in this survey is briefly discussed below.

4.1. *Dense Storage Packages*

LAPACK is a standard library of routines for solving the most common problems in numerical linear algebra (Anderson et al. 1994), which relies on the BLAS (Dongarra et al. 1989) routines for most of its operations. The results of this paper were generated by using vendor-optimized versions of the BLAS, which typically increase the performance of the LAPACK routines several times over the use of the generic, nonoptimized BLAS routines. Routine DGETRF is called to compute the lower-upper (LU) factorization of the matrix $\tilde{A}$ using partial pivoting with row interchanges. Routine DGETRS is then called to solve a system of linear equations $\tilde{A} \cdot x = b$ and repeated for as many right-hand sides as necessary.

LUDCMP is the name of the routine described in Press et al. (1996). The full matrix $\tilde{A}$ is replaced by its LU decomposition using partial pivoting with implicit scaling. Routine LUBKSB is then called to solve a system of linear equations $\tilde{A} \cdot x = b$. Calls to LUBKSB are repeated for as many right-hand sides as required.

LEQS is a routine that solves a system of linear equations by Gaussian elimination. The origin of this legacy routine is somewhat obscure; it was in use at least by 1962 (J. W. Truran 1999, private communication) and is probably the most common linear algebra package presently used for evolving reaction networks. The matrix $\tilde{A}$ is reduced to upper triangular form in tandem with a right-hand side b by Gaussian elimination, and back-substitution on the upper triangular matrix yields the solution to $\tilde{A} \cdot x = b$. The maximum element in each row serves as the pivot element, but no row or column interchanges are performed, so LEQS may become unstable if used on matrices that are not diagonally dominant. A small amount of effort is devoted to minimizing calculations with matrix elements that are zero. This routine, like all Gaussian elimination routines, has the disadvantage that when there is a sequence of right-hand sides whose entries depend on a previous member in the sequence (as in eqs. [11] and [12]–[14]), the entire matrix must be decomposed for each right-hand side in the sequence.

GIFT is a program that generates FORTRAN subroutines for solving a system of linear equations by Gaussian elimination (Gustafson, Liniger, & Willoughby 1970; Müller 1997). The full matrix $\tilde{A}$ is reduced to upper triangular form, and back-substitution with the right-hand side b yields the solution to $\tilde{A} \cdot x = b$. GIFT-generated routines skip all calculations with matrix elements that are zero; in this restricted sense, GIFT-generated routines are sparse, but the storage of a full matrix is still required. It is assumed that the pivot element is located on the diagonal and no row or column interchanges are performed, so GIFT-generated routines may become unstable if the matrices are not diagonally dominant. This routine, like the Gaussian elimination routine LEQS, must decompose the matrix for each right-hand side in a staged set of linear equations. GIFT writes out (in FORTRAN code) the sequence of Gaussian elimination and back-substitution without any

loop constructions on the matrix $A(i, j)$. As a result, the routines generated by GIFT can be quite large. For the 200 isotope reaction network shown in Figure 6, GIFT generated about 7.6×10^4 lines of code. For the 489 isotope reaction network, GIFT generated 4.9×10^5 lines of code when the matrix was put in the up-arrow configuration and about 5.0×10^7 lines when the matrix was put in the down-arrow configuration.

4.2. *Sparse Storage Packages*

There are two methods for solving sparse linear systems of equations: direct and iterative. Direct methods typically obtain the solution to $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} = \boldsymbol{b}$ by LU decomposition or Gaussian elimination and back-substitution. Direct methods obtain the solution to a set of linear equations in a finite, well-determined number of operations. Thus, direct methods are fairly easy to characterize. Iterative (or "matrix-free") methods seek to minimize a function whose gradient is typically $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} - \boldsymbol{b}$ and equal to zero when the function is minimized. These methods are attractive because they tend to require only matrix-times-vector operations and usually have smaller storage requirements than direct methods have. However, the number of iterations required to converge to a solution is not known a priori, and the iterations count generally increases with the number of unknowns. The total number of iterations, hence the overall speed, depends crucially on the initial guess and on the stringency of the convergence criteria. While this survey focuses on matrix packages that use direct methods for solving sparse linear equations, one matrix package that uses an iterative method is included.

Direct methods for sparse matrices typically divide the solution of $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} = \boldsymbol{b}$ into a symbolic LU decomposition, numerical LU decomposition, and a back-substitution phase. In the symbolic LU decomposition phase the pivot order of a matrix is determined, and a sequence of decomposition operations that minimize the amount of fill-in is recorded. Fill-in refers to zero matrix elements that become nonzero (e.g., a sparse matrix times a sparse matrix is generally a denser matrix). The matrix is not (usually) decomposed; only the steps to do so are stored. Since the nonzero pattern of a chosen nuclear reaction network does not change, the symbolic LU decomposition is a one-time initialization cost for reaction networks. In the numerical LU decomposition phase, a matrix with the same pivot order and nonzero pattern as a previously factorized matrix is numerically decomposed into its LU form. This phase must be done only once for each staged set of linear equations. In the back-substitution phase, a set of linear equations is solved with the factors calculated from a previous numerical decomposition. The back-substitution phase may be performed with as many right-hand sides as needed, and not all of the right-hand sides need to be known in advance. All of the sparse-matrix packages used in this survey accept the nonzero entries of the matrix in the $(i, j, a_{i,j})$ coordinate system, and all of the sparse-matrix packages use 70%–90% less storage than their dense-matrix counterparts.

MA28 is described in Duff, Erisman, & Reid (1986). It uses a combination of nested dissection and frontal envelope decomposition to minimize fill-in during the factorization stage. An approximate degree update algorithm that is much faster (asymptotically and in practice) than computing the exact degrees is employed. One continuous real parameter sets the amount of searching done to locate the pivot element. When this parameter is set to zero, no searching is done and the diagonal element is the pivot, while when it is set to unity, complete partial pivoting is done. Since the matrices generated by reaction networks are diagonally dominant, the routine was set to use the diagonal as the pivot element. Several test cases showed that using partial pivoting did not make a significant difference in accuracy, but was less efficient since a search for an appropriate pivot element had to be performed.

UMFPACK is described by Davis & Duff (1997). It uses a combined unifrontal and multifrontal method to factorize a sparse matrix by using a sequence of small dense frontal matrices. These dense frontal matrices are themselves factorized using the BLAS dense-matrix kernels. Again, the results of this paper were generated by using vendor-optimized versions of the BLAS routines. The pivot-search strategy is almost the same as that used in the MA28 package.

Y12M is described by Zlatev, Wasniewski, & Schaumburg (1981). Nested dissection techniques are used during the symbolic LU decomposition phase, and a Markowitz strategy is used to identify the pivot element. Y12M has an optional "drop tolerance," where any matrix element that is less than the drop tolerance is pruned from the tree structure. This reduces the total number of matrix elements, and thus will usually achieve better efficiency. Drop tolerances were, however, not used in this survey of nuclear reaction networks.

BiCG is described by Barrett et al. (1993). This method, which is just one of many iterative methods, generates a sequence of vectors for the matrix $\tilde{\boldsymbol{A}}$ and another sequence for the transpose matrix $\tilde{\boldsymbol{A}}^T$. These vector sequences are the residuals of the iterations. These two sequences are made mutually orthogonal, or bi-orthogonal. In this paper, the preconditioner matrix was taken to be the tridiagonal part of the reaction networks Jacobian matrices (Figs. 1–7), which tended to give slightly faster convergence properties than simply taking the diagonal of the matrix (Jacobi preconditioning). A convergence criterion of $|\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} - \boldsymbol{b}|/|\boldsymbol{b}| < 10^{-12}$ was used for all the reaction networks.

5. RESULTS

The timing tests were run on seven different serial computers, six UNIX workstations, and one LINUX PC. Each of the serial machines had a different CPU clock speed (180–450 MHz), bus clock speed (30–200 Mbyte s^{-1}), and cache memory size (0.512–4 Mbyte). Each of the 168 combinations (7 reaction networks, 3 integration methods, 8 linear algebra packages) was compiled under FORTRAN 77 and FORTRAN 90. When possible, the compilation was performed with one of six different compiler option sets: from a set that requested no code optimization to a set that requested that routines be in-lined, that do-loops be unrolled,and that aggressive code optimization be used. The absolute speed of each combination depends, obviously, on the machine architecture and compiler options employed. These dependences can be minimized, and meaningful comparisons made, by examining the relative speeds of the various combinations. For this reason, the results of the timing tests shown in Figures 8–14 have been normalized to the average CPU time consumed by an appropriate combination for the type comparison being analyzed. However, the absolute CPU time consumed by one particular

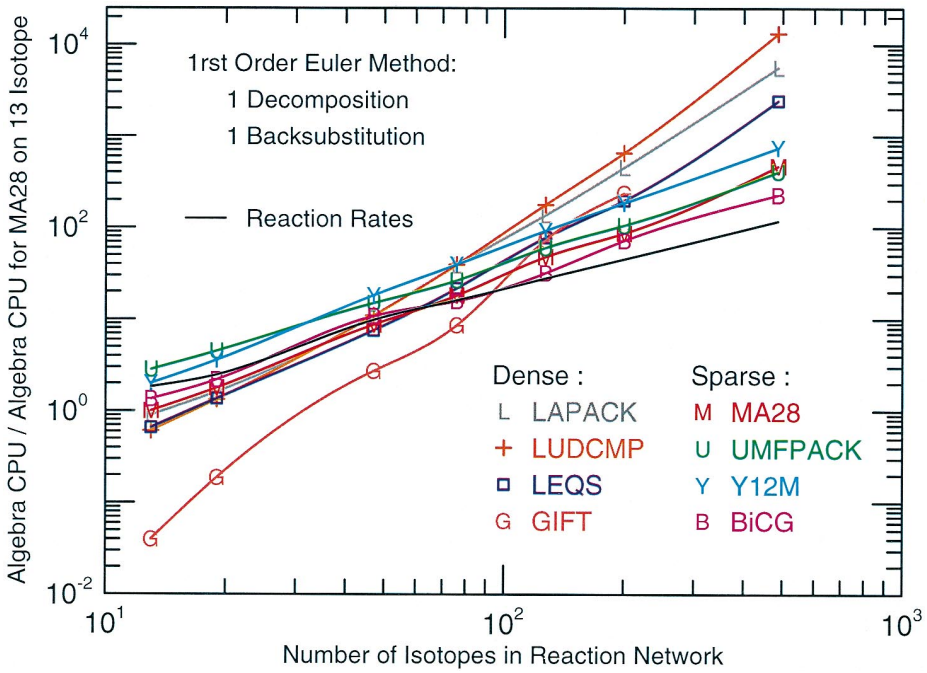

FIG. 8.—CPU time of the linear algebra for a single time step with the first-order–accurate Euler method. The CPU time is normalized to the CPU time for the 13 isotope reaction network with MA28 package and the Euler method in order to minimize the dependence on machine architecture and compiler options. Each of the linear algebra packages is given a different symbol and color. All of the dense-matrix packages are more efficient in execution speed than the three sparse-matrix packages when there are less than about 60 isotopes in the reaction network. In this case, however, evaluation of the nuclear reaction rates dominates the total cost of a single time step.

machine and one particular set of compiler options is given in Tables 1–3. The average CPU times were determined by calling each of the 168 combinations a total of 10^6 times. The total CPU time for each combination was then divided by 10^6 to obtain the average CPU time per call.

5.1. *Cost per Time Step, across Linear Algebra Packages*

The CPU time consumed by the linear algebra for a single time step with the first-order–accurate Euler method is shown in Figure 8. The number of isotopes in the reaction

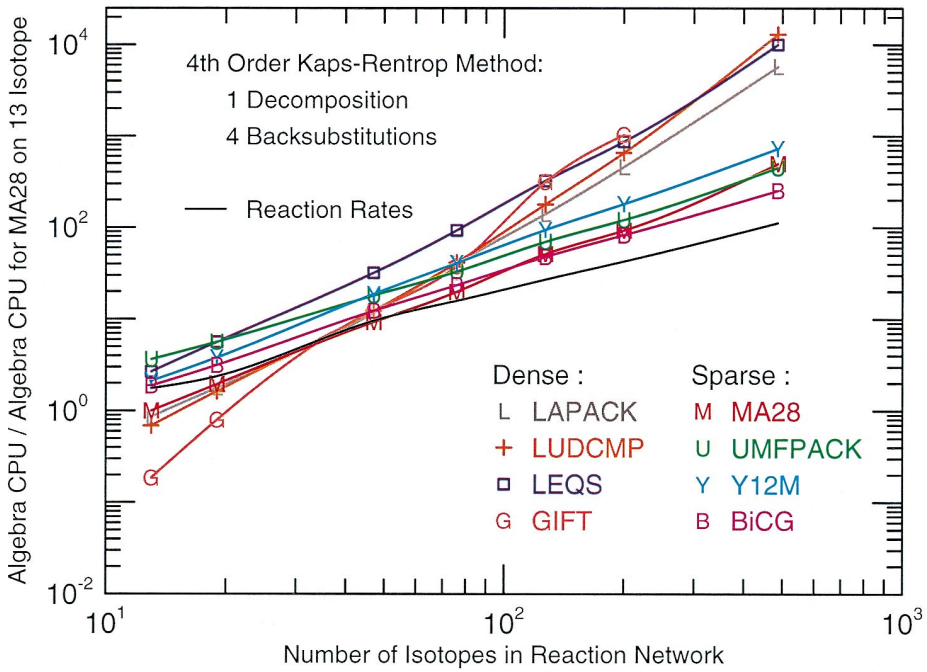

FIG. 9.—CPU time of the linear algebra for a single, successful time step with the fourth-order accurate Kaps-Rentrop method. A successful time step is one that meets a given integration accuracy. The CPU time is normalized to the CPU time for the 13 isotope reaction network with MA28 package and the Kaps-Rentrop method. Otherwise, the format is the same as the format used in Fig. 8. The sparse-matrix packages are more efficient in execution speed than the dense-matrix packages when there are more than about 60 isotopes in the reaction network. The relative cost of evaluating the nuclear reaction rates (*solid black line*) is smaller with the Kaps-Rentrop method since there are more linear algebra operations per time step. Linear algebra packages that do not treat decomposition and back-substitution separation lose efficiency with higher order integration methods.

TABLE 1

CPU Time per Time Step for First-Order Euler Method[a]

Number of Isotopes (1)	Total Time (2)	Rate		Integrate		Algebra	
		Time (3)	% (4)	Time (5)	% (6)	Time (7)	% (8)
LAPACK:							
13	0.20	0.10	47.64	0.06	30.20	0.05	22.16
19	0.29	0.13	44.73	0.08	27.54	0.08	27.73
47	1.23	0.48	39.29	0.21	16.95	0.54	43.75
76	3.16	0.82	26.03	0.42	13.33	1.91	60.63
127	9.21	1.45	15.75	0.96	10.39	6.80	73.86
200	27.16	2.36	8.70	2.33	8.60	22.47	82.70
487	309.40	6.05	1.96	23.88	7.72	279.50	90.33
LUDCMP:							
13	0.18	0.09	50.92	0.06	32.50	0.03	16.58
19	0.27	0.13	46.69	0.08	28.74	0.07	24.57
47	1.23	0.48	38.81	0.21	16.95	0.54	44.24
76	3.21	0.84	26.29	0.42	12.98	1.95	60.74
127	11.19	1.46	13.03	0.96	8.54	8.77	78.43
200	37.01	2.38	6.43	2.34	6.31	32.29	87.26
487	689.90	6.06	0.88	23.92	3.47	659.90	95.65
LEQS:							
13	0.19	0.09	48.93	0.06	33.33	0.03	17.74
19	0.27	0.12	45.49	0.08	29.60	0.07	24.92
47	1.05	0.47	44.21	0.21	19.97	0.38	35.83
76	2.28	0.78	34.14	0.42	18.48	1.08	47.37
127	6.17	1.37	22.21	0.94	15.27	3.86	62.52
200	14.45	2.28	15.80	2.41	16.69	9.76	67.51
487	152.90	6.09	3.98	25.78	16.86	121.00	79.16
GIFT:							
13	0.16	0.09	58.91	0.06	39.84	0.00	1.25
19	0.22	0.13	56.62	0.09	39.19	0.01	4.19
47	0.90	0.51	56.67	0.25	28.17	0.14	15.15
76	1.81	0.88	48.54	0.51	27.93	0.43	23.52
127	7.10	1.79	25.24	1.68	23.71	3.63	51.05
200	18.35	2.65	14.45	4.01	21.86	11.69	63.70
MA28:							
13	0.21	0.09	44.36	0.07	31.49	0.05	24.15
19	0.29	0.12	42.54	0.08	26.89	0.09	30.57
47	1.11	0.49	44.23	0.18	16.69	0.43	39.08
76	2.00	0.80	40.11	0.30	15.08	0.90	44.81
127	4.32	1.40	32.30	0.56	12.97	2.36	54.74
200	7.49	2.26	30.14	0.96	12.76	4.28	57.10
487	31.77	5.88	18.52	2.75	8.65	23.14	72.83
UMF:							
13	0.30	0.10	31.90	0.06	20.99	0.14	47.11
19	0.43	0.13	29.52	0.08	18.23	0.23	52.24
47	1.42	0.48	33.91	0.19	13.11	0.75	52.97
76	2.44	0.80	32.89	0.31	12.64	1.33	54.47
127	4.97	1.39	27.96	0.56	11.22	3.03	60.82
200	8.51	2.23	26.20	0.98	11.49	5.30	62.31
487	29.10	5.92	20.33	2.81	9.66	20.38	70.01
Y12M:							
13	0.26	0.09	36.48	0.06	23.84	0.10	39.68
19	0.39	0.13	32.87	0.08	20.12	0.18	47.01
47	1.58	0.48	30.16	0.18	11.55	0.92	58.29
76	3.09	0.80	25.86	0.30	9.77	1.99	64.37
127	6.58	1.38	20.96	0.55	8.40	4.65	70.64
200	12.41	2.18	17.57	0.94	7.61	9.29	74.82
487	45.74	5.64	12.33	2.78	6.07	37.32	81.60
BiCG:							
13	0.22	0.09	41.96	0.06	27.79	0.07	30.24
19	0.31	0.13	40.31	0.08	24.96	0.11	34.72
47	1.20	0.48	39.73	0.19	15.76	0.54	44.51

TABLE 1—*Continued*

Number of Isotopes (1)	Total Time (2)	Rate Time (3)	Rate % (4)	Integrate Time (5)	Integrate % (6)	Algebra Time (7)	Algebra % (8)
76	1.88	0.79	42.22	0.31	16.29	0.78	41.49
127	3.49	1.36	38.98	0.55	15.79	1.58	45.23
200	6.61	2.16	32.70	0.89	13.42	3.56	53.88
487	19.37	5.46	28.19	2.70	13.95	11.21	57.86

[a] For one Silicon Graphics 250 MHz processor, R10010 floating-point chip, 4 Mbyte data cache, 200 MHz s^{-1} bus, and a compiler line of "f90 -O2 -IPA." All the CPU times are given in milliseconds. Col. (1) lists the linear algebra package and the number of isotopes in the reaction network, col. (2) the average total CPU time, col. (3) the average CPU time for evaluating the nuclear reaction rates, col. (4) the percentage of the total time used for evaluating the reaction rates, col. (5) the average CPU time for the integration method, col. (6) the percentage of the total time used by the integration method, col. (7) the average CPU time used by the linear algebra, and col. (8) the percentage of the total time used by the linear algebra. This data set is representative of the data used to generate the figures.

network forms the x-axis. The CPU time, normalized to the CPU time for the 13 isotope reaction network with the MA28 package and the Euler method, forms the y-axis. Each of the linear algebra packages is assigned a different symbol and color. The relative cost of evaluating the nuclear reaction rates is shown by the solid black line.

Recall from § 3 that a single time step taken with the first-order Euler method (which has no assessment of the accuracy of the time step) requires one matrix decomposition and one back-substitution. The cost of evaluating the Jacobian matrix and the right-hand side are not included in Figure 8, since attention is focused on the performance of the linear algebra packages. The relative cost of including formation of the right-hand side and Jacobian matrix is discussed below.

Figure 8 shows that all of the dense solvers (LAPACK, LUDCMP, LEQS, and GIFT) are more efficient in execution speed than the sparse solvers (MA28, UMFPACK, and Y12M) when there are less than about 60 isotopes in the reaction network. In this case, however, evaluation of the nuclear reaction rates dominates the total cost of a single time step. Each nuclear reaction rate is a sum of exponential and power functions, which are much more expensive to evaluate than the additions and multiplications done by the linear algebra. When there are more than about 60 isotopes in the reaction network, the sparse-matrix solvers are more efficient in execution speed than the dense-matrix solvers for a single time step—by more than an order of magnitude for the 489 isotope reaction network. Any of the sparse-matrix packages requires much less memory storage than any of the dense-matrix packages for any network size, since the matrices associated with nuclear reaction networks have a large fraction of zero matrix elements.

Some care must be given to interpreting the results for the BiCG matrix package. The overall efficiency of any iterative method is sensitive to the spectral properties of the coefficient matrix, the initial guess, and stringency of the convergence criteria. In Figure 8, a tridiagonal preconditioner was used and the initial guess for the solution was always set to zero. Using only the diagonal of the matrix, or a solution from the previous set of staged linear equations, increased the number of iterations and hence would increase the CPU time shown in Figure 8.

The GIFT generated routines (one set of routines for each reaction network) are particularly efficient on the smaller reaction networks when used with the first-order Euler method. They execute about 25 times faster than any of the other linear algebra packages on the 13 isotope reaction network and about 3 times faster than any others on the 47 isotope reaction network, and are about as fast on the 200 isotope reaction network as the sparse-matrix packages MA28, UMFPACK, and Y12M. There is no data point for the GIFT package operating on the 489 isotope reaction network because GIFT generated so much FORTRAN code (4.9×10^5 lines in the up-arrow configuration, 5.0×10^7 lines in the down-arrow configuration) that the routines could not easily be compiled on any of the machines used in this survey.

The CPU time required by the linear algebra for a single, successful time step with the fourth-order–accurate Kaps-Rentrop method is shown in Figure 9. The CPU time, normalized to the CPU time for the 13 isotope reaction network with the MA28 package and the Kaps-Rentrop method, forms the y-axis. Otherwise, the format of Figure 9 is the same as the format of Figure 8. A successful time step is defined as a time step that meets or exceeds a specified integration accuracy. Recall from § 3 that a single, successful time step taken with the fourth-order Kaps-Rentrop method nominally requires one matrix decomposition and four back-substitutions.

As in Figure 8, the dense-matrix solvers are more efficient in execution speed than the sparse-matrix solvers for less than about 60 isotopes in the reaction network. For more than 60 isotopes in the reaction network, the sparse-matrix solvers are faster (by 2 orders of magnitude for the 489 isotope reaction network) than the dense-matrix solvers for a single time step.

In contrast to Figure 8, Figure 9 shows that the relative cost of evaluating the nuclear reaction rates (Fig. 9 *solid black curve*) is smaller, since the Kaps-Rentrop method has a larger amount of linear algebra per step than the Euler method. The absolute cost of evaluating the nuclear reaction rates is, of course, the same for any time integration method. Figure 9 shows that the relative cost of evaluating the nuclear reaction rates is about the same as the relative cost of performing the linear algebra for less than about 50 isotopes in the reaction network. For more than 50 isotopes in the reaction network, the relative cost of evaluating the nuclear reaction rates is small (less than 1% for the 489 isotope reaction network).

TABLE 2

CPU Time per Time Step for Fourth-Order Kaps-Rentrop Method[a]

Number of Isotopes (1)	Total Time (2)	Rate		Integrate		Algebra	
		Time (3)	% (4)	Time (5)	% (6)	Time (7)	% (8)
LAPACK:							
13	0.29	0.09	31.62	0.16	52.84	0.05	15.54
19	0.41	0.13	31.14	0.18	45.49	0.09	23.37
47	1.46	0.47	31.91	0.38	25.66	0.62	42.43
76	3.61	0.80	22.09	0.66	18.43	2.15	59.48
127	10.34	1.46	14.11	1.47	14.19	7.42	71.70
200	30.70	2.46	8.03	4.00	13.02	24.24	78.96
487	335.50	5.97	1.78	27.03	8.05	302.50	90.16
LUDCMP:							
13	0.27	0.10	35.30	0.14	51.41	0.04	13.29
19	0.38	0.13	33.48	0.17	43.75	0.09	22.78
47	1.48	0.48	32.38	0.35	23.62	0.65	44.00
76	3.61	0.81	22.39	0.62	17.10	2.19	60.51
127	12.18	1.46	11.99	1.32	10.88	9.40	77.13
200	40.73	2.48	6.10	3.82	9.38	34.42	84.53
487	718.20	6.12	0.85	27.07	3.77	685.10	95.38
LEQS:							
13	0.36	0.09	25.54	0.13	35.49	0.14	38.97
19	0.59	0.13	21.45	0.17	28.06	0.30	50.49
47	2.57	0.49	18.96	0.40	15.41	1.69	65.64
76	6.54	0.84	12.77	0.80	12.23	4.91	75.00
127	20.84	1.57	7.52	2.21	10.60	17.06	81.88
200	54.83	2.56	4.66	6.45	11.77	45.82	83.57
487	578.50	6.18	1.07	43.14	7.46	529.10	91.47
GIFT:							
13	0.24	0.09	39.48	0.13	56.37	0.01	4.15
19	0.33	0.13	37.43	0.17	50.01	0.04	12.56
47	1.55	0.52	33.34	0.38	24.36	0.66	42.31
76	3.59	0.89	24.74	0.71	19.82	1.99	55.44
127	20.51	1.81	8.82	2.23	10.85	16.48	80.33
200	61.47	2.69	4.38	4.63	7.54	54.14	88.08
MA28:							
13	0.29	0.09	31.97	0.15	49.85	0.05	18.19
19	0.40	0.13	31.69	0.17	42.29	0.10	26.02
47	1.32	0.51	38.38	0.32	24.43	0.49	37.19
76	2.37	0.84	35.40	0.48	20.36	1.05	44.24
127	4.97	1.43	28.87	0.83	16.16	2.71	54.51
200	8.49	2.26	26.65	1.34	15.82	4.89	57.53
487	36.11	5.95	16.47	4.07	11.27	26.09	72.26
UMF:							
13	0.44	0.10	22.21	0.15	33.70	0.20	44.09
19	0.61	0.13	21.31	0.17	28.82	0.30	49.87
47	1.78	0.49	27.49	0.33	18.29	0.97	54.22
76	3.08	0.83	26.87	0.49	15.88	1.76	57.25
127	6.00	1.44	23.91	0.85	14.11	3.72	61.98
200	10.13	2.25	22.19	1.44	14.22	6.44	63.60
487	33.97	5.80	17.08	4.17	12.27	24.00	70.65
Y12M:							
13	0.35	0.10	27.56	0.14	40.16	0.11	32.28
19	0.50	0.13	25.97	0.17	33.16	0.20	40.87
47	1.79	0.48	26.60	0.31	17.19	1.01	56.21
76	3.46	0.80	23.15	0.47	13.55	2.19	63.30
127	7.24	1.39	19.24	0.82	11.34	5.03	69.43
200	13.29	2.29	17.24	1.36	10.22	9.64	72.54
487	48.68	5.91	12.15	3.88	7.96	38.89	79.89
BiCG:							
13	0.31	0.09	29.79	0.12	38.38	0.10	31.83
19	0.44	0.13	28.98	0.15	33.03	0.17	37.99
47	1.41	0.47	33.40	0.29	20.46	0.65	46.14
76	2.47	0.78	31.72	0.45	18.16	1.24	50.12
127	4.65	1.35	29.08	0.77	16.63	2.52	54.29
200	7.70	2.12	27.48	1.23	15.93	4.36	56.59
487	22.19	5.34	24.08	3.58	16.14	13.27	59.79

[a] The machine architecture, compiler options, and column layout are the same as in Table 1.

TABLE 3

MINIMUM CPU TIME PER TIME STEP FOR VARIABLE-ORDER BADER-DEUFLHARD METHOD[a]

NUMBER OF ISOTOPES (1)	TOTAL TIME (2)	RATE		INTEGRATE		ALGEBRA	
		Time (3)	% (4)	Time (5)	% (6)	Time (7)	% (8)
LAPACK:							
13	0.51	0.10	18.74	0.28	55.67	0.13	25.59
19	0.68	0.13	18.92	0.33	48.15	0.22	32.93
47	2.42	0.49	20.16	0.61	25.35	1.32	54.49
76	6.36	0.83	13.10	1.11	17.54	4.41	69.37
127	18.91	1.50	7.95	2.32	12.26	15.09	79.78
200	58.64	2.47	4.21	6.45	10.99	49.73	84.79
487	671.00	6.10	0.91	43.56	6.49	621.30	92.60
LUDCMP:							
13	0.45	0.09	20.66	0.29	64.63	0.07	14.71
19	0.64	0.13	20.07	0.33	52.46	0.17	27.47
47	2.46	0.47	19.11	0.63	25.44	1.37	55.46
76	6.50	0.81	12.44	1.11	17.02	4.58	70.54
127	23.17	1.47	6.33	2.33	10.08	19.37	83.59
200	78.86	2.45	3.11	6.39	8.11	70.01	88.78
487	1422.00	6.00	0.42	43.92	3.09	1372.00	96.49
LEQS:							
13	0.73	0.09	12.92	0.28	39.12	0.35	47.97
19	1.20	0.13	10.76	0.34	28.01	0.73	61.23
47	5.36	0.52	9.76	0.65	12.19	4.19	78.05
76	14.33	0.88	6.12	1.17	8.13	12.29	85.75
127	46.93	1.57	3.34	2.66	5.67	42.70	90.99
200	134.50	2.56	1.91	7.30	5.43	124.70	92.67
487	1475.00	6.15	0.42	44.62	3.02	1425.00	96.56
GIFT:							
13	0.41	0.09	21.68	0.32	76.57	0.01	1.76
19	0.59	0.12	20.46	0.38	64.36	0.09	15.17
47	2.81	0.50	17.84	0.77	27.36	1.54	54.79
76	7.15	0.90	12.52	1.37	19.21	4.88	68.27
127	45.78	1.80	3.94	4.10	8.96	39.87	87.09
200	145.70	2.70	1.86	8.31	5.70	134.70	92.44
MA28:							
13	0.50	0.09	18.73	0.30	58.96	0.11	22.31
19	0.68	0.13	18.85	0.34	49.33	0.22	31.83
47	2.09	0.47	22.31	0.57	27.39	1.05	50.30
76	3.85	0.80	20.86	0.85	22.00	2.20	57.14
127	8.61	1.40	16.21	1.46	16.96	5.75	66.83
200	14.62	2.19	14.97	2.24	15.32	10.19	69.72
487	63.47	5.82	9.17	6.42	10.12	51.23	80.71
UMF:							
13	0.83	0.10	11.56	0.30	36.65	0.43	51.79
19	1.14	0.13	11.39	0.35	30.40	0.66	58.22
47	3.13	0.47	14.95	0.64	20.33	2.02	64.72
76	5.36	0.78	14.62	0.95	17.74	3.63	67.65
127	10.72	1.38	12.86	1.63	15.18	7.72	71.96
200	18.42	2.24	12.18	2.62	14.22	13.56	73.60
487	61.80	5.85	9.46	7.74	12.52	48.21	78.02
Y12M:							
13	0.56	0.09	16.71	0.26	46.70	0.20	36.58
19	0.79	0.13	15.80	0.30	37.66	0.37	46.53
47	2.75	0.47	17.11	0.53	19.33	1.75	63.56
76	5.41	0.79	14.54	0.81	15.03	3.81	70.43
127	10.98	1.35	12.31	1.41	12.85	8.21	74.84
200	20.06	2.14	10.67	2.27	11.31	15.65	78.02
487	75.40	5.70	7.56	6.42	8.51	63.29	83.93
BiCG:							
13	0.59	0.09	15.88	0.25	43.34	0.24	40.78
19	0.82	0.13	15.39	0.29	35.82	0.40	48.79
47	2.56	0.47	18.49	0.54	21.00	1.55	60.52
76	4.57	0.79	17.30	0.81	17.72	2.97	64.97
127	8.77	1.38	15.70	1.40	15.91	6.00	68.40
200	14.70	2.16	14.67	2.19	14.91	10.35	70.41
487	41.38	5.48	13.25	5.93	14.34	29.96	72.41

[a] The machine architecture, compiler options, and column layout are the same as in Table 1.

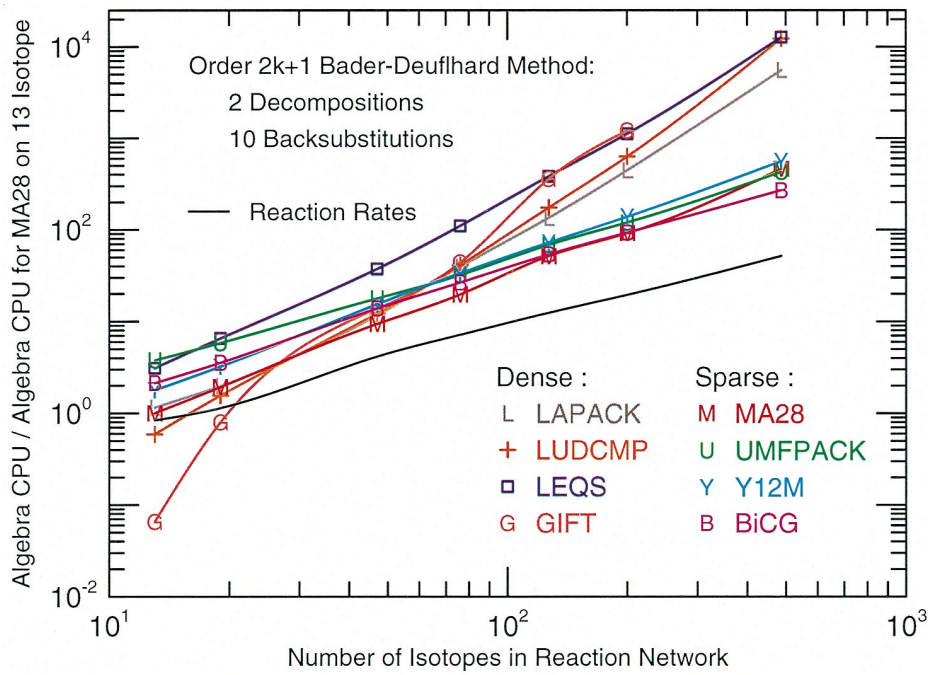

FIG. 10.—Minimum CPU time of the linear algebra for a single, successful time step with the variable-order Bader-Deuflhard method. The CPU time is normalized to the CPU time for the 13 isotope reaction network with MA28 package and the Bader-Deuflhard method. The format is the same as in Fig. 8. The relative cost of evaluating the nuclear reactions is insignificant since the Bader-Deuflhard method has a comparatively large number of linear algebra operations associated with it. If the Bader-Deuflhard method takes accurate time steps that are comparatively large, however, then the method will be more efficient globally.

Higher order integration routines, such as the Kaps-Rentrop method, typically require the solutions to a staged set of linear equations (Δ_4 depends on $\Delta_3 \ldots$ depends on Δ_1, as in equation [11]). These higher order integration routines will reward matrix packages that first decompose the matrix (the expensive part) and then solve as many staged sets of linear equations by back-substitution (the inexpensive part) as necessary. Matrix packages that must decompose the matrix for every staged set of linear equations will be penalized under higher order integration methods. This explains why the relative performance of the LEQS and the GIFT packages are degraded in Figure 9 as compared to Figure 8. Both of these dense-matrix packages, which rely on Gaussian elimination to bring the matrix into upper triangular form, must solve each staged set of linear equations a priori. The fourth-order Kaps-Rentrop method nominally requires only three more back-substitutions than the first-order Euler method, but this gets turned into three additional decompositions with the LEQS and GIFT matrix packages. Thus, instead of being about 25 times faster than any of the other linear algebra packages on the 13 isotope reaction network (Fig. 8), the GIFT-generated routines are now about six times faster (Fig. 9). Instead of being three times faster on the 47 isotope network, the GIFT routines are now no faster than any other matrix package. All of the sparse-matrix packages are faster than the GIFT routines when there are more than 100 isotopes in the reaction network and the Kaps-Rentrop method is used. Similar efficiency losses occur for the Gaussian elimination package LEQS.

The minimum CPU time required for a single, successful time step with the variable-order Bader-Deuflhard method is shown in Figure 10. The CPU time is normalized to the CPU time for the 13 isotope reaction network with the MA28 package and the Bader-Deuflhard method. Normalization is performed to decrease, but not eliminate, the dependence on machine architecture and compiler options. The format of Figure 10 is the same as the format in Figure 8. It bears repeating that a successful time step is defined as a time step that meets or exceeds a specified integration accuracy. Recall from § 3 that a single, successful time step taken with the variable-order Bader-Deuflhard method requires a minimum of two matrix decompositions and 10 back-substitutions per time step. This minimum requirement can increase at a rate of one decomposition and m back-substitutions for every increase in the order k. The cost of increasing the order is compensated for by taking a corresponding larger (but accurate) time step, and the controls for achieving this goal are part of the Bader-Deuflhard method.

As in Figures 8 and 9, the crossover point where the sparse, matrix packages become more efficient than the dense-matrix packages is about 60 isotopes in the reaction network. The sparse-matrix packages, of course, require far less storage than the dense-matrix packages for any network size. In contrast to Figures 8 and 9, Figure 10's relative cost of evaluating the nuclear reactions is insignificant, since the Bader-Deuflhard method has much more linear algebra associated with it.

As a higher order integration method, the Bader-Deuflhard method rewards those matrix packages that break the solution of $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} = \boldsymbol{b}$ into a matrix decomposition phase and a back-substitution phase. The reward is nearly the same size as for the Kaps-Rentrop method (four back-substitutions for every matrix decomposition), since the Bader-Deuflhard method has a minimum of five back-substitutions per matrix decomposition per successful time step. Accordingly, the relative efficiency loss of the LEQS and the GIFT packages is about the same in Figure 10 as it is in Figure 9 (both compared to Fig. 8). However, the

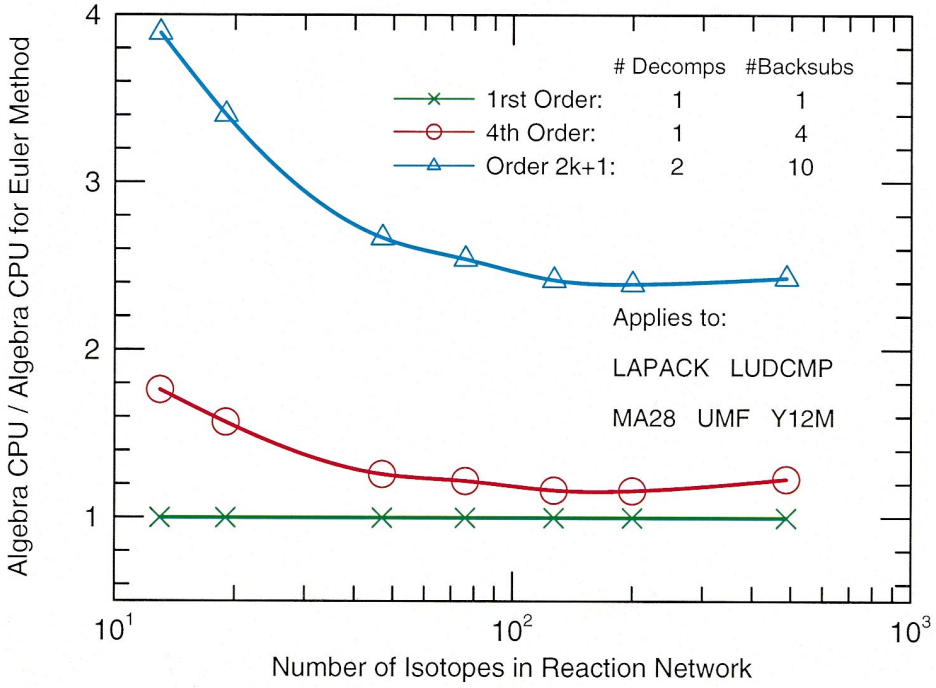

FIG. 11.—CPU time required by the linear algebra for a single, successful time step across the three time integration methods. The CPU time for each time integration method is normalized to the CPU time required for the first-order Euler method. Each of the integration methods has been assigned a different symbol and a color, and this figure applies to the linear algebra packages LAPACK, LUDCMP, MA28, UMFPACK, and Y12M. These matrix packages decompose the matrix once, and back-substitute for as many staged sets of linear equations as needed. The cost of using the Kaps-Rentrop or the minimal Bader-Deuflhard method with these linear algebra packages is about 10% and 2.5 times more, respectively, than using the Euler method for more than about 50 isotopes. Since the Euler method provides no estimate of the accuracy of a time step, either of the higher order integration methods is worth the extra cost.

minimum cost per time step of the Bader-Deuflhard method is at least twice as large as the cost per step of the Kaps-Rentrop method or the Euler method. In order to be more efficient globally, the Bader-Deuflhard method must be able to take accurate time steps that are at least twice as large.

The slopes in Figures 8–10 indicate that the LAPACK and LUDCMP dense-matrix packages approach the expected scaling relation CPU time $\sim N^{3.0}$, where N is the number of isotopes in the reaction network. The LEQS package has a smaller scaling exponent that approaches 2.8. This exponent is smaller because the LEQS package takes some advantage of zero matrix elements. More erratic scaling behavior is shown by the GIFT package, but appears to be approaching an exponent of 2.7. The sparse-matrix packages MA28, UMFPACK, and Y12M have scaling exponents that are near 1.8, 1.5, and 1.7, respectively. Thus, the UMFPACK sparse-matrix package will consume a smaller amount of CPU than MA28 or Y12M for a sufficiently large number of isotopes. Figures 8–10 suggest that the crossover point where the UMFPACK package is more efficient has already been reached for the 489 isotope reaction network.

Placing neutrons, protons, and α-particles first in the soft-wired networks puts the Jacobian matrix in the up-arrow configuration, while placing the neutrons, protons, and α-particles last in the isotope list puts the Jacobian matrix in the down-arrow configuration. While the choice of up-arrow or down-arrow configurations makes no different physically, placing the Jacobian matrix in the up-arrow configuration dramatically increases the amount of fill-in that occurs during the matrix decomposition phase for the dense-matrix packages LAPCK, LUDCMP, and LEQS. The much larger amount of fill-in results in CPU times that are about eight times longer for the 47 isotope reaction network and about 200 times longer for the 489 isotope reaction network. Thus, the neutrons, protons, and α-particles should always be placed last in the isotope list with these dense-matrix packages. The opposite conclusion holds for the GIFT package. Typically GIFT generates 8–100 times more FORTRAN code when the Jacobian matrices are placed in the down-arrow configuration. For example, GIFT generated about 4.9×10^5 lines of code for the 489 isotope network in the up-arrow configuration and 5.0×10^7 lines of FORTRAN when the 489 isotope network was placed in the down-arrow configuration. Whether the Jacobian matrix is placed in the up-arrow or down-arrow configuration makes less than 1% difference in execution speed with the MA28, UMFPACK, or Y12M sparse-matrix packages, since each employs algorithms to minimize the amount of fill-in during the elimination phase. Hence, either orientation may be used with these packages for the direct solution of sparse linear equations.

5.2. *Cost per Time Step, across Time Integration Methods*

Figures 8–10 compare the execution efficiency of the various matrix packages for each of the three time integration methods. Figures 11 and 12 show the situation from an orthogonal direction by comparing the execution efficiency of the time integration methods for a given matrix package. While Figures 11 and 12 duplicate some of the information contained in Figures 8–10, they provide additional insights by presenting the information from another viewpoint.

The CPU time consumed by the linear algebra for a single, successful time step using the three integration methods is shown in Figure 11. The number of isotopes in the reaction network forms the abscissa, and the CPU time

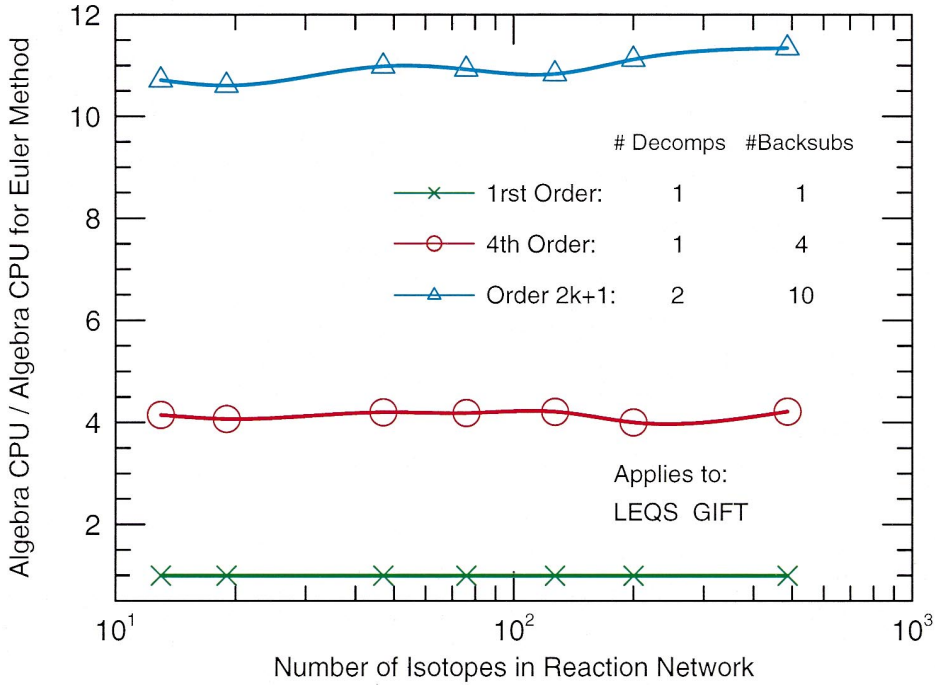

FIG. 12.—CPU time required by the linear algebra for a single, successful time step across the three time integration methods. The CPU time for each time integration method is normalized to the CPU time required for the first-order Euler method, and the format is the same as in Fig. 11. This figure applies to the LEQS and GIFT dense-matrix packages, both of which must decompose the matrix for every staged set of linear equations. The nominal 4 back-substitutions associated with the Kaps-Rentrop method requires 4 decompositions with these matrix packages, and thus the Kaps-Rentrop method is 4 times less efficient than the Euler method with these matrix packages. Similarly, the minimal Bader-Deuflhard method is 11 times less efficient than the Euler method with these matrix packages.

normalized to the CPU time for the first-order Euler method forms the oridinate. Each of the integration methods is assigned a different symbol and a color, as indicated in the legend. Figure 11 applies to the LAPACK, LUDCMP, MA28, UMFPACK, and Y12M matrix packages, all of which first decompose the matrix and then solve as many staged sets of linear equations as necessary through back-substitution. The execution speed per time step of the fourth-order Kaps-Rentrop method is within 80% of the execution speed per time step of the first-order

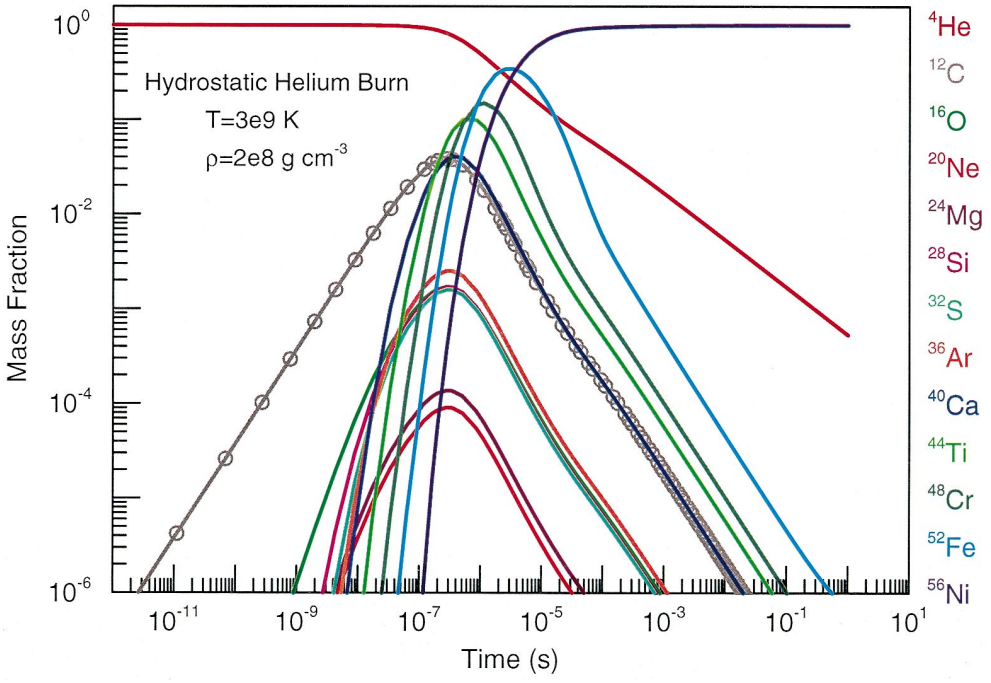

FIG. 13.—Evolution of the abundance levels during hydrostatic helium burning. Time is on the x-axis, and the mass fraction of an isotope is given on the y-axis. An initially pure helium composition (*solid red line*) in the 13 isotope reaction network is exposed to a temperature of 3×10^9 K and a density of 2×10^8 g cm^{-3} for 1 s. Each abundance curve is colored according to the legend at the right of the figure. The final ashes are mainly ^{52}Fe with a small admixture of ^{56}Ni. The time points where the solution was calculated by this integration are shown as the open gray circles on the ^{12}C mass fraction.

Euler method for the 13 isotope reaction network and within 60% for the 19 isotope reaction network. For the larger reaction networks used in this survey, the cost of using the Kaps-Rentrop method with these linear algebra packages is only about 10% more per time step than using the Euler method. Since the Euler method provides no estimate of the accuracy of a time step, while the Kaps-Rentrop method does, the small additional cost per step incurred by using the Kaps-Rentrop method is well worth the effort. The minimum execution speed per time step of the variable-order Bader-Deuflhard method is about four times slower than the first-order Euler method for the 13 isotope reaction network and about 3.4 times slower for the 19 isotope reaction network. The Bader-Deuflhard method approaches an asymptotic limit of 2.5 times slower per time step than the Euler method, because the Bader-Deuflhard method must complete a minimum of two matrix decompositions per successful time step. The Bader-Deuflhard also returns an estimate of the time step accuracy, but the minimum cost per time step of this estimate is larger than with the Kaps-Rentrop method.

The CPU time consumed by the LEQS and GIFT matrix packages for a single, successful time step is shown in Figure 12. The legend and format of this figure is the same as in Figure 11. The point made previously about using two matrix packages with higher-order integration routines is illustrated quite well by Figure 12. These two dense-matrix packages must, as all Gaussian elimination packages must, decompose the matrix for every staged set of linear equations that need to be solved. That is, not all the right-hand sides are known in advance. The nominal four back-substitutions associated with the Kaps-Rentrop method require four decompositions with these matrix packages, and thus the Kaps-Rentrop method is four times slower than the Euler method with these matrix packages in Figure 12. For similar reasons, the Bader-Deuflhard method is 11 times less efficient than the Euler method with these matrix packages. Figures 11 and 12 strongly suggest that if higher order integration methods are to be used (which they should, because they provide time-step accuracy estimates), then one should also use a matrix package that permits the efficient solution to a staged set of linear equations.

5.3. *Total Cost across Many Time Steps*

In the two previous sections the analysis of the time integration methods and linear algebra packages concentrated on the relative costs per time step. In this section the analysis will focus on a test case where many time steps are required and the total cost of the evolution is analyzed.

Many cases were examined during the course of this survey: hydrostatic hydrogen to hydrostatic silicon burning, explosive hydrogen to explosive silicon burning, postprocessing of exploding white dwarfs and massive stars, and incorporation of the nuclear reaction networks into serial and parallel versions of the Center on Astrophysical Thermonuclear Flashes three-dimensional hydrodynamic program, FLASH. Only the results from hydrostatic helium burning will be reported in this paper. This test vehicle was chosen because it is simple yet broad enough to cover a wide range of stellar phenomena, and it is representative of the results found for the other cases.

An initially pure helium composition at a constant temperature of 3×10^9 K and a constant density of 2×10^8 g cm^{-3} was evolved for 1 s. These conditions can occur in multidimensional models of X-ray bursts. Evolution of the abundance levels for the 19 isotope network is shown in Figure 13. The time is given on the x-axis, and the mass fraction of an isotope is given on the y-axis. Some of the

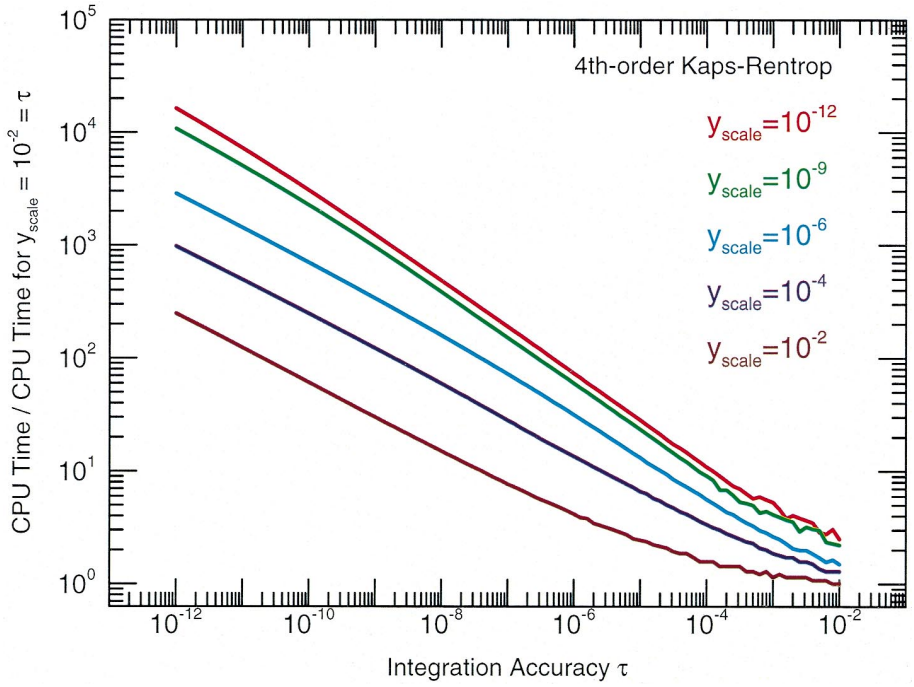

FIG. 14.—Total CPU time consumed by the fourth-order Kaps-Rentrop method for the hydrostatic helium-burning test case with the 19 isotope reaction network. The integration accuracy τ forms the x-axis, and the CPU time normalized to the CPU time for the $\tau = 10^{-2}$, $y_{\rm scale} = 10^{-2}$ integration forms the y-axis. Each colored curve corresponds to a constant abundance level $y_{\rm scale}$ which is used in scaling the error estimates. The cost of the Kaps-Rentrop method increases linearly (on logarithmic scales) with the integration accuracy for all choices of the scale factor $y_{\rm scale}$.

abundances are labeled and colored according to the legend given to the right of the figure. The time points where the solution was calculated by this integration are shown as the open circles on the ^{12}C mass fraction (*gray solid curve*). Starting from a pure helium composition (*red curve*) the final products, for the chosen thermodynamic conditions, are chiefly ^{56}Ni (*purple curve*).

5.3.1. *Total Cost across Integration Methods*

The total CPU time consumed by the fourth-order Kaps-Rentrop method in integrating the hydrostatic helium-burning test case is shown in Figure 14. The integration accuracy τ forms the x-axis, and the CPU time normalized to the CPU time for the $\tau = 10^{-2}$, $y_{\rm scale} = 10^{-2}$ integration forms the y-axis. Each colored curve in Figure 14 corresponds to a constant abundance level $y_{\rm scale}$ used to scale the error estimates. When an abundance is greater than $y_{\rm scale}$, a relative error is calculated, while for abundances smaller than $y_{\rm scale}$, the absolute error is calculated (Press et al. 1996). In essence, only abundances greater than $y_{\rm scale}$ can exert control on the size of the time step. The cost of the Kaps-Rentrop method increases linearly (on the logarithmic scales of Figure 14) with the integration accuracy for all choices of the scale factor $y_{\rm scale}$. At a constant integration accuracy, the CPU time increase is nearly proportional to the scale factor. When the scale factor is smaller than about 10^{-9}, the total CPU time becomes nearly independent of the scale factor.

The total CPU time consumed by the variable-order Bader-Rentrop method in integrating the hydrostatic helium-burning test case is shown in Figure 15. Again, the CPU time is normalized to the CPU time for the $\tau = 10^{-2}$, $y_{\rm scale} = 10^{-2}$ integration, and the format of the figure is the same as in Figure 14. The cost of the Bader-Deuflhard method increases linearly (on logarithmic scales) with the integration accuracy for scale factors less than about 10^{-6}. The slope of the curves in Figure 15, however, is much smaller than the slope for the curves for the Kaps-Rentrop method. When the scale factor is smaller than about 10^{-9}, the total CPU time used by the Bader-Deuflhard method becomes nearly independent of the scale factor.

The first-order Euler method is excluded from these global efficiency tests, since this method provides no estimates on the accuracy of a given time step. One can, of course, use some heuristics in order to integrate a given reaction network. Such heuristics are typically based upon limiting the change in any abundance that is above some level to less than some specified value (say, 10%). While nuclear reaction networks can be integrated in this manner, and commonly are, one has no formal estimate of how accurate the integration steps are.

The total CPU time consumed by the fourth-order Kaps-Rentrop method relative to the variable-order Bader-Deuflhard for the hydrostatic helium-burning test case is shown in Figure 16. The format of the figure is the same format used in Figure 14. For modest accuracy and scaling choices ($\tau \sim 10^{-4}$, $y_{\rm scale} \sim 10^{-4}$), the two integration methods have comparable efficiencies. For more restrictive accuracy and scaling requirements, the Bader-Deuflhard method is more than an order of magnitude more efficient. Thus, even though this method is at least a factor of 2 times more expensive per time step, it can take large enough time steps to become more efficient globally. The decline in the curves in Figure 16 for the scalings $y_{\rm scale} \lesssim 10^{-9}$ indicates that the Bader-Deuflhard method loses some of its relative efficiency over the Kaps-Rentrop method for this hydrostatic helium-burning test case when very strict integration tolerance of $\tau \lesssim 10^{-4}$ are imposed.

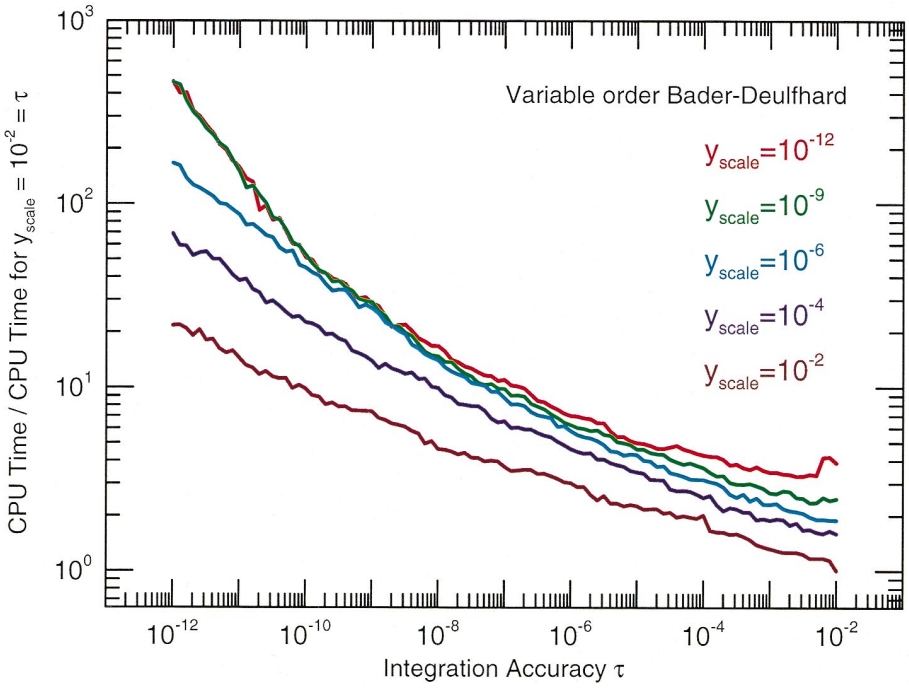

FIG. 15.—Total CPU time consumed by the variable-order Bader-Deuflhard for the hydrostatic helium-burning test case with the 19 isotope reaction network. The format is the same as in Fig. 14. The cost of the Bader-Deuflhard method increases linearly (on logarithmic scales) with the integration accuracy for relatively large choices of the scale factor $y_{\rm scale}$. The total CPU time is nearly constant for $y_{\rm scale} \lesssim 10^{-6}$.

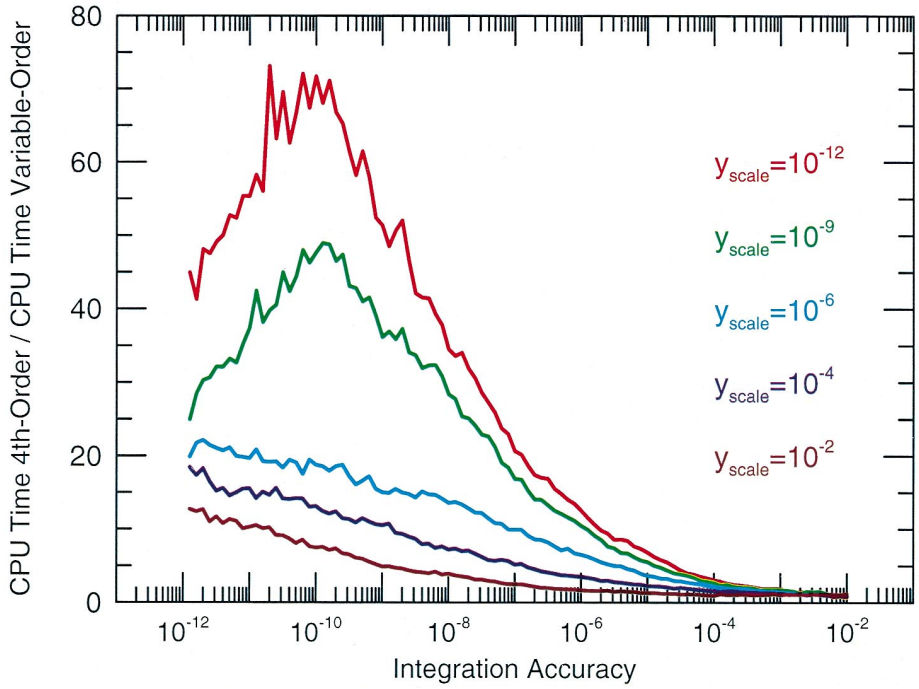

FIG. 16.—Total CPU time consumed by the fourth-order Kaps-Rentrop method relative to the variable-order Bader-Deuflhard method for the hydrostatic helium-burning test case with the 19 isotope reaction network. The format is the same as in Fig. 14. For modest accuracy and scaling choices ($\tau \sim 10^{-4}$, $y_{scale} \sim 10^{-4}$), the two methods are comparable. For more restrictive accuracy and scaling requirements, the Bader-Deuflhard method is more than an order of magnitude more efficient.

Figures 14–16 indicate that the variable-order Bader-Deuflhard method is generally as efficient as, and often much more efficient than, the fourth-order Kaps-Rentrop method when the hydrostatic helium-burning test case is evolved with the 19 isotope network. As the number of isotopes increases, the Bader-Deuflhard method becomes even more efficient compared with the Kaps-Rentrop method.

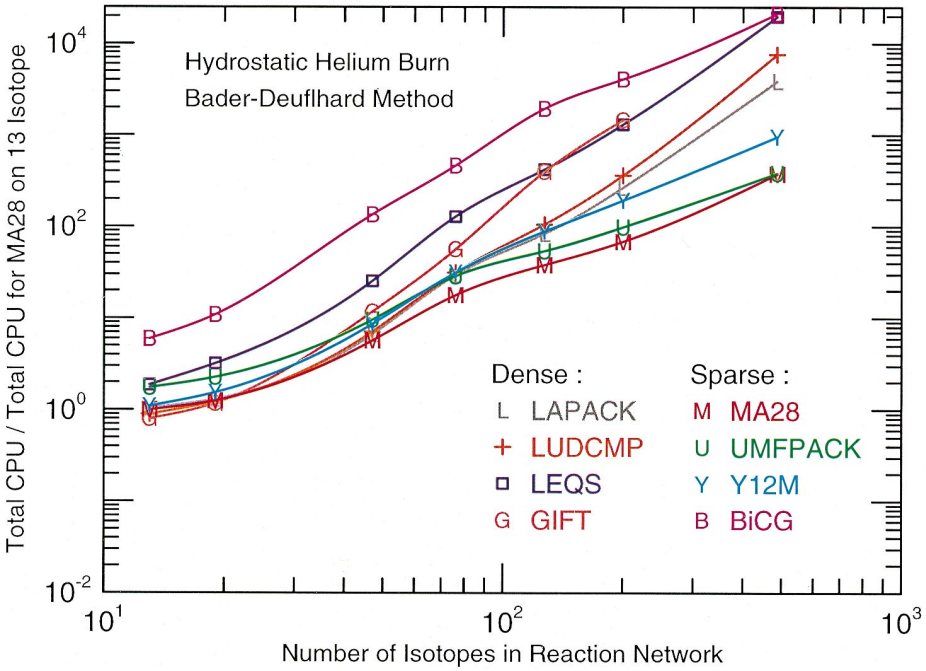

FIG. 17.—Total CPU time consumed by the variable-order Bader-Deuflhard method for the hydrostatic helium-burning test case. The total CPU time is normalized to the total CPU time for the 13 isotope reaction network with the MA28 package; otherwise the format is the same as in Fig. 8. The MA28 sparse-matrix package is generally more efficient than the other matrix packages for this test case, and similar efficiencies were found for other test cases.

TABLE 4

TOTAL CPU TIME FOR HYDROSTATIC HELIUM-BURNING TEST CASE WITH THE BADER-DEUFLHARD METHOD[a]

NUMBER OF ISOTOPES (1)	TOTAL TIME (2)	RATE		INTEGRATE		ALGEBRA	
		Time (3)	% (4)	Time (5)	% (6)	Time (7)	% (8)
LAPACK:							
13	48.92	0.13	0.26	34.25	70.01	14.54	29.73
19	61.62	0.14	0.22	37.13	60.27	24.35	39.51
47	315.50	0.53	0.17	109.00	34.56	205.90	65.27
76	1389.0	0.87	0.06	319.40	23.00	1068.0	76.94
127	3920.0	1.63	0.04	561.60	14.33	3357.0	85.63
200	12500.0	2.50	0.02	1424.0	11.40	11070.0	88.58
487	182200.0	5.66	0.0	10130.0	5.56	172100.0	94.44
LUDCMP:							
13	42.08	0.12	0.28	33.29	79.11	8.67	20.61
19	57.80	0.14	0.23	36.89	63.82	20.78	35.94
47	335.20	0.52	0.15	109.10	32.55	225.60	67.30
76	1477.0	0.91	0.06	310.00	20.99	1166.0	78.95
127	4936.0	1.65	0.03	548.40	11.11	4386.0	88.86
200	16930.0	2.57	0.02	1434.0	8.47	15490.0	91.52
487	357900.0	6.43	0.0	10230.0	2.86	347700.0	97.14
LEQS:							
13	88.22	0.12	0.13	33.28	37.73	54.82	62.14
19	151.20	0.14	0.09	36.32	24.02	114.70	75.89
47	1199.0	0.60	0.05	119.50	9.97	1079.0	89.98
76	6023.0	1.00	0.02	347.40	5.77	5675.0	94.22
127	19720.0	1.65	0.01	670.40	3.40	19050.0	96.59
200	61430.0	2.60	0.0	1711.0	2.79	59720.0	97.21
487	931400.0	5.85	0.0	10500.0	1.13	920900.0	98.87
GIFT:							
13	37.59	0.12	0.33	30.97	82.39	6.49	17.28
19	55.30	0.13	0.23	36.04	65.17	19.13	34.60
47	552.90	0.61	0.11	140.10	25.33	412.20	74.55
76	2652.0	1.02	0.04	413.10	15.58	2238.0	84.38
127	18570.0	1.72	0.01	1098.0	5.91	17470.0	94.08
200	68470.0	2.57	0.00	2057.0	3.00	66410.0	96.99
MA28:							
13	46.90	0.13	0.28	37.45	79.84	9.32	19.88
19	59.25	0.15	0.24	39.60	66.83	19.51	32.93
47	268.00	0.56	0.21	107.60	40.15	159.80	59.64
76	837.30	0.86	0.10	275.50	32.91	560.90	66.99
127	1793.0	1.52	0.08	433.70	24.19	1357.0	75.72
200	3219.0	2.45	0.08	708.90	22.02	2507.0	77.90
487	17500.0	6.01	0.03	2587.00	14.78	14910.0	85.19
UMF:							
13	83.45	0.12	0.15	36.09	43.25	47.24	56.60
19	108.30	0.15	0.14	38.89	35.91	69.24	63.95
47	458.60	0.55	0.12	109.70	23.92	348.30	75.96
76	1369.0	0.93	0.07	290.50	21.22	1078.0	78.71
127	2569.0	1.52	0.06	466.50	18.16	2101.0	81.79
200	4675.0	2.37	0.05	782.20	16.73	3890.0	83.22
487	18130.0	5.98	0.03	2690.0	14.84	15430.0	85.13
Y12M:							
13	52.34	0.12	0.23	33.26	63.54	18.96	36.22
19	73.70	0.15	0.20	36.03	48.88	37.52	50.92
47	412.0	0.56	0.14	102.30	24.82	309.20	75.04
76	1502.0	0.90	0.06	270.60	18.02	1230.0	81.92
127	4203.0	1.53	0.04	437.20	10.40	3764.0	89.56
200	9144.0	2.41	0.03	714.70	7.82	8427.0	92.16
487	45460.0	5.93	0.01	2472.00	5.44	42990.0	94.55
BiCG:							
13	280.40	0.12	0.04	32.61	11.63	247.60	88.33
19	514.0	0.15	0.03	35.51	6.91	478.30	93.06
47	6327.0	0.62	0.01	103.20	1.63	6223.0	98.36
76	21690.0	0.98	0.0	270.70	1.25	21420.0	98.75
127	91260.0	1.64	0.0	442.80	0.49	90810.0	99.51
200	191900.0	2.55	0.0	723.10	0.38	191200.0	99.62

[a] The machine architecture, compiler options, and column layout are the same as in Table 1.

5.3.2. *Total Cost across Linear Algebra Packages*

The total CPU time consumed by the Bader-Deuflhard method in evolving the hydrostatic helium-burning test case is shown in Figure 17. All seven nuclear reaction networks executed the test case with an integration accuracy of $\tau = 10^{-5}$ and a constant scaling factor of $y_{\rm scale} = 10^{-6}$, and the number of isotopes in the reaction network forms the x-axis. Each of the linear algebra packages is assigned a different symbol and color, as indicated by the legend. The total CPU time is normalized to the total CPU time for the 13 isotope reaction network with the MA28 package on the y-axis. Normalization minimizes, but does not completely eliminate, the dependence on machine architecture and compiler options. The absolute CPU time consumed, on average, by one particular machine and one particular set of compiler options is given in Table 4.

When there are less than about 60 isotopes in the reaction network, the LAPACK, LUDCMP, GIFT, MA28, UMFPACK, and Y12M all have comparable efficiencies. When there are more than about 100 isotopes in the network, Figure 17 shows that the MA28, UMFPACK, and Y12M packages integrate the hydrostatic helium-burning test case faster than the other matrix packages used in this survey. The BiCG matrix package performed the worst in this test case because convergence stagnated, which required a large number of iterations to finally reach convergence. Modifying the initial guess at each time step and using a pentadiagonal preconditioner were tried, but neither of these modifications significantly improved the rate of convergence. The LEQS matrix package also performed poorly in this test case, since each back-substitution must become a matrix decomposition with the staged set of linear equations that appear in high-order integration methods. For the same reason, the GIFT package did not perform well when there were more than 60 isotopes in the reaction network, even though the GIFT matrix package did perform slightly better than any other matrix package for the 13 isotope reaction network.

Figure 17 indicates that the MA28 sparse-matrix package is generally more efficient than the other matrix packages when evolving the hydrostatic helium-burning test case. Similar efficiencies were found for other burning scenarios, such as explosive carbon burning and multidimensional models of X-ray bursts.

6. SUMMARY AND SUGGESTIONS

The execution speed of three semi-implicit time integration methods operating with eight linear algebra packages have been examined on seven nuclear reaction networks. The results of this survey (the figures and tables) permit a few pragmatic suggestions to be made.

The first-order Euler method for integrating nuclear reaction networks should be abandoned, since the method provides no time step accuracy estimates. Abundance sets calculated using the first-order Euler method should be viewed with caution, unless perhaps the abundance levels are demonstrated to be robust with respect to the size of the time steps.

Higher order integration routines that depend on the solution to a staged set of linear equations, such as the fourth-order Kaps-Rentrop method and the variable-order Bader-Deuflhard method examined in this paper, should be coupled with a matrix package that splits the solution of $\tilde{\boldsymbol{A}} \cdot \boldsymbol{x} = \boldsymbol{b}$ into a matrix decomposition phase and a back-substitution phase. Matrix packages that rely on Gaussian elimination, such as LEQS or GIFT, should be avoided with higher order integration methods, since neither operates efficiently when there are more than 50 isotopes.

If the execution speed of small reaction networks (fewer than about 60 isotopes) is an overriding concern, then the variable-order Bader-Deuflhard time integration method coupled with routines generated from the GIFT matrix package or LAPACK with vendor-optimized BLAS routines is a good choice. If the amount of computer memory required is a concern for any size reaction network, then using any of the sparse-matrix packages (MA28, UMFPACK, Y12M, or BiCG) will reduce the storage required by 70%–90%. When a balance between accuracy, overall efficiency, and ease of use is desirable, the variable-order Bader-Deuflhard time integration method coupled with the MA28 sparse-matrix package is a very good choice. For reaction networks that contain more isotopes than do the ones used in this survey, replacing the MA28 sparse-matrix package with the UMFPACK sparse-matrix package may improve the overall efficiency.

These suggestions are enthusiastically endorsed whether the nuclear reaction networks are run in stand-alone mode, in stellar evolution programs, in postprocessing the thermodynamic histories of stellar models, or in multidimensional hydrodynamic programs. In particular, by invoking a full ordinary differential equation integrator inside of a multidimensional hydrodynamics program that employs operator splitting, one is assured of the accuracy of the reaction network integrations. In addition, any explicit subcycling of the reaction networks with the hydrodynamics is avoided since the integrators will automatically "subcycle" to integrate to the desired time point with the requested accuracy.

In multidimensional hydrodynamic programs with nuclear burning, the reaction network must be integrated in thousands to millions of zones. The reaction network integrations should be performed in vector or parallel mode. Some of the time integration methods and linear algebra packages discussed in this survey are difficult to execute efficiently on vector or parallel architectures. Some of the linear algebra packages, such as GIFT, can here achieve impressive speed increases on vector architectures (E. Müller 1999, private communication). Some of the integration methods and packages, such as the Bader-Deuflhard method and the MA28 sparse-matrix package, can achieve a decent speed increase on parallel machines with efficient processor communication software. A key point in getting maximum performance out of putting each zone's reaction network on a separate processor is to get maximum performance out of the serial software. This is one reason that this survey focused on the efficiency of the time integration methods and linear algebra packages on serial architectures.

Finally, the results of this survey apply only to the case of nuclear reaction networks. The choice of an integration method for stiff equations, and particularly the choice of a linear algebra package, is highly dependent on the problem type and the nonzero pattern of the matrices.

This work is supported by the Department of Energy under grant B341495 to the Center on Astrophysical Thermonuclear Flashes at the University of Chicago. The author thanks David Arnett, Rob Hoffman, Stan Woosley,

and Jim Truran for many conversations over the last decade on nuclear reaction networks, and particularly Rob Hoffman for keeping the author's nuclear reaction rate database current. The author also thanks the anonymous referee and Raph Hix for prompt, constructive reviews. The various time integration methods and linear algebra packages used in this survey may be obtained by contacting the author.

REFERENCES

Anderson, E., et al. 1994, LAPACK 2.0 Users Guide (Philadelphia: SIAM)
Arnett, D. 1996, Supernovae and Nucleosynthesis: An Investigation of the History of Matter, from the Big Bang to the Present (Princeton: Princeton Univ. Press)
Bader, G., & Deuflhard, P. 1983, Numer. Math., 41, 373
Barrett, R., et al. 1993, Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods (Philadelphia: SIAM)
Burgers, J. M. 1969, Flow Equations for Composite Gases (New York: Academic)
Chapman, S., & Cowling, T. G. 1970, The Mathematical Theory of Non-uniform Gases (Cambridge: Cambridge Univ. Press)
Davis, T. A., & Duff, I. S. 1997, Tech. Rep. TR-97-016, Comput. Inf. Sci. Dept. Univ. Florida
Dongarra, J., DuCroz, J., Hammarling, S., & Hanson, R. 1989, ACM Trans. Math. Software, 14, 1
Duff, I. S., Erisman, A. M., & Reid, J. K. 1986, Direct Methods for Sparse Matrices (Oxford: Oxford Univ. Press)
Gustafson, F. G., Liniger, W., & Willoughby, R. 1970, J. Assoc. Comput. Mach., 17, 87
Kaps, P., & Rentrop, P. 1979, Numer. Math., 33, 55
Müller, E. 1997, in Saas-Fée Advanced Course 27, Computational Methods for Astrophysical Fluid Flow, ed. O. Steiner, & A. Gautschy (Berlin: Springer), 343
Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. 1996, Numerical Recipes in FORTRAN 90 (Cambridge: Cambridge Univ. Press)
Weaver, T. A., Zimmerman, G. B., & Woosley, S. E. 1978, ApJ, 225, 1021
Williams, F. A. 1988, Combustion Theory (Menlo Park: Benjamin-Cummings)
Woosley, S. E., & Hoffman, R. D. 1992, ApJ, 395, 202
Woosley, S. E., & Weaver, T. A. 1995, ApJS, 101, 181
Zlatev, Z., Wasniewski, J., & Schaumburg, K. 1981, Y12M: Solution of Large and Sparse Systems of Linear Algebraic Equations (Berlin: Springer)